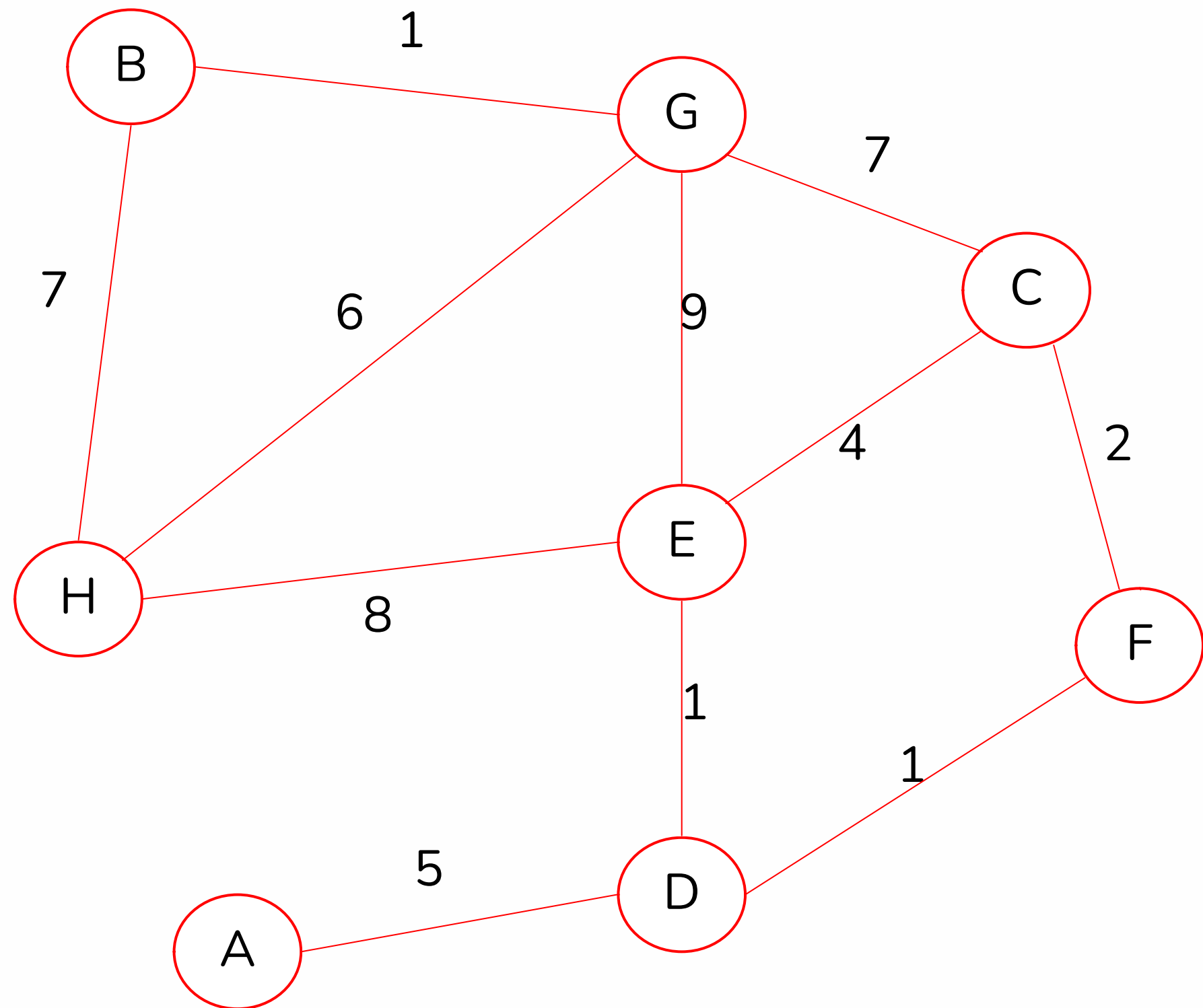


BFS - DIJKSTRA BELLMAN FORD

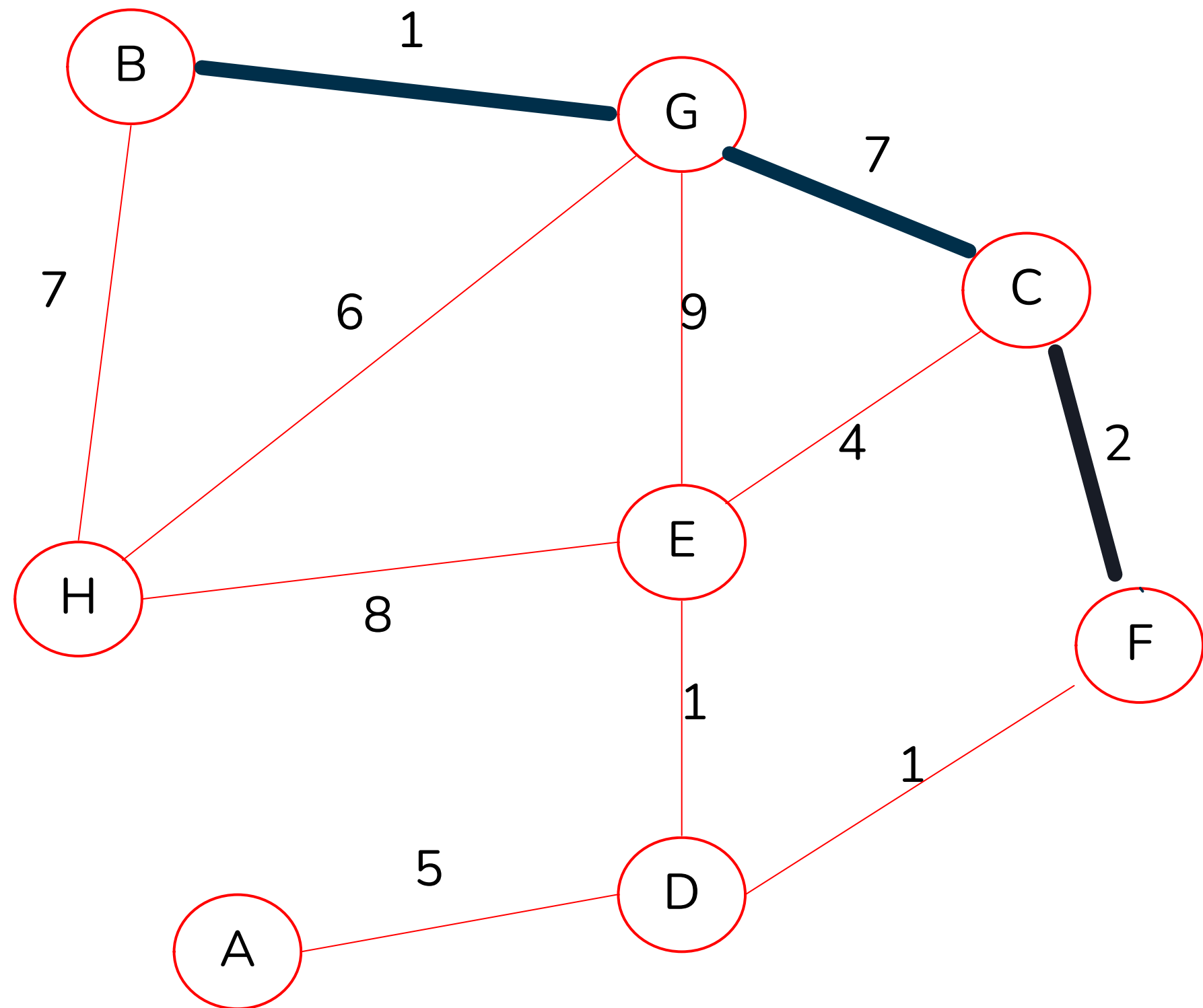
Joaquín Peralta - Elías Ayaach - Alex Infanta - Steven Reynolds - Paula Grune

EL CAMINO MÁS CORTO



Definimos el camino más corto entre dos nodos como la menor suma de las aristas que los conectan

EL CAMINO MÁS CORTO



Por ejemplo, el camino más corto entre B y F

¿Cómo podemos encontrar este camino más corto?

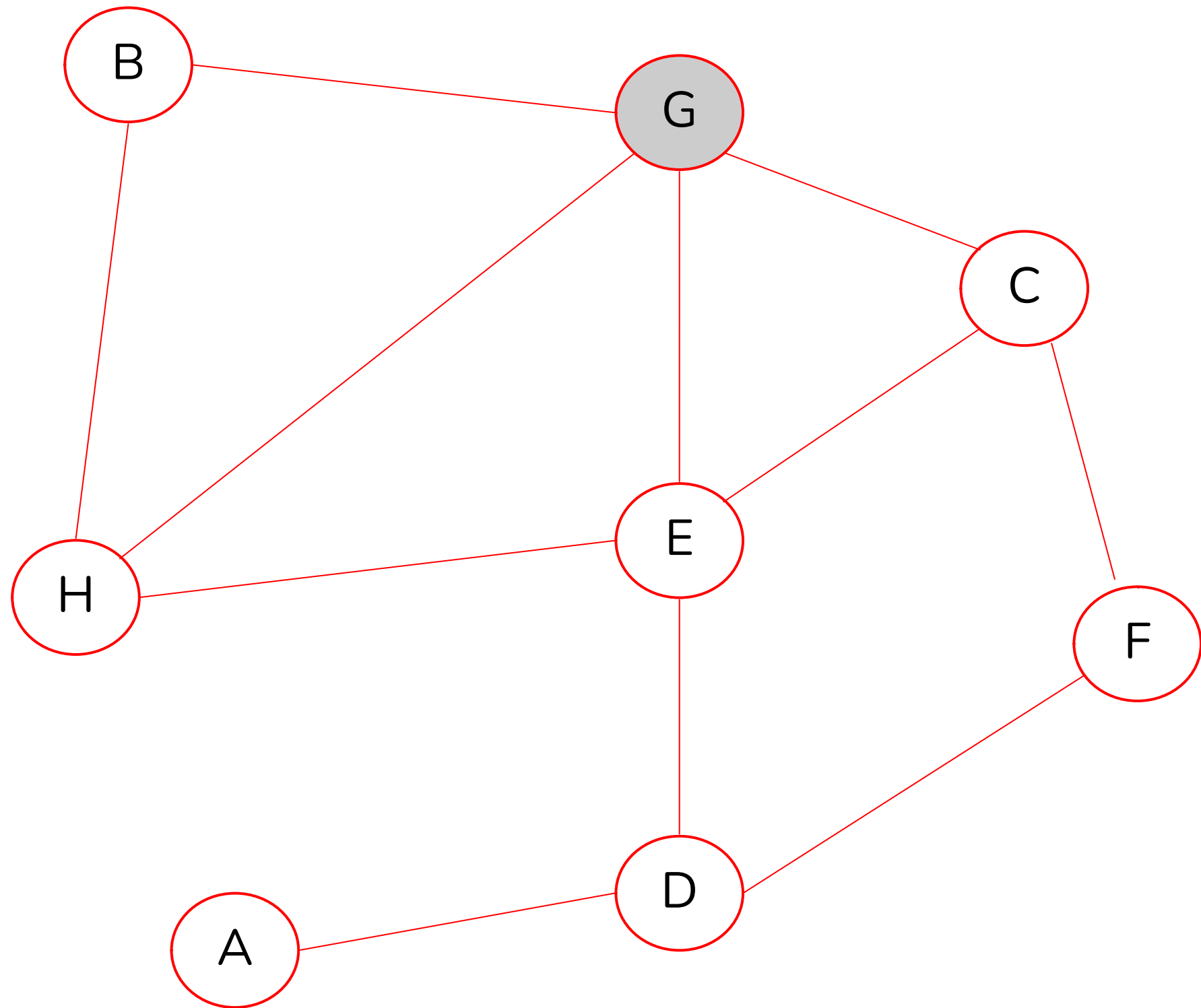
BBREADTH
FFIRST
SSEARCH

BFS

Antes de ver cómo encontramos el camino más corto, debemos entender como funciona BFS:

- Es un algoritmo de **búsqueda en amplitud**.
- Partiendo de un nodo del grafo, recorreremos primero los nodos que están a una arista de distancia, luego a dos aristas de distancia, y así hasta llegar al último nodo.
- Marcamos los nodos que ya visitamos con el fin de no revisar infinitamente.
- Para esto **usamos una cola FIFO**, cada vez que encontramos un nodo nuevo lo agregamos al final de la cola y para revisar nuevos sacamos el primer nodo.

BFS

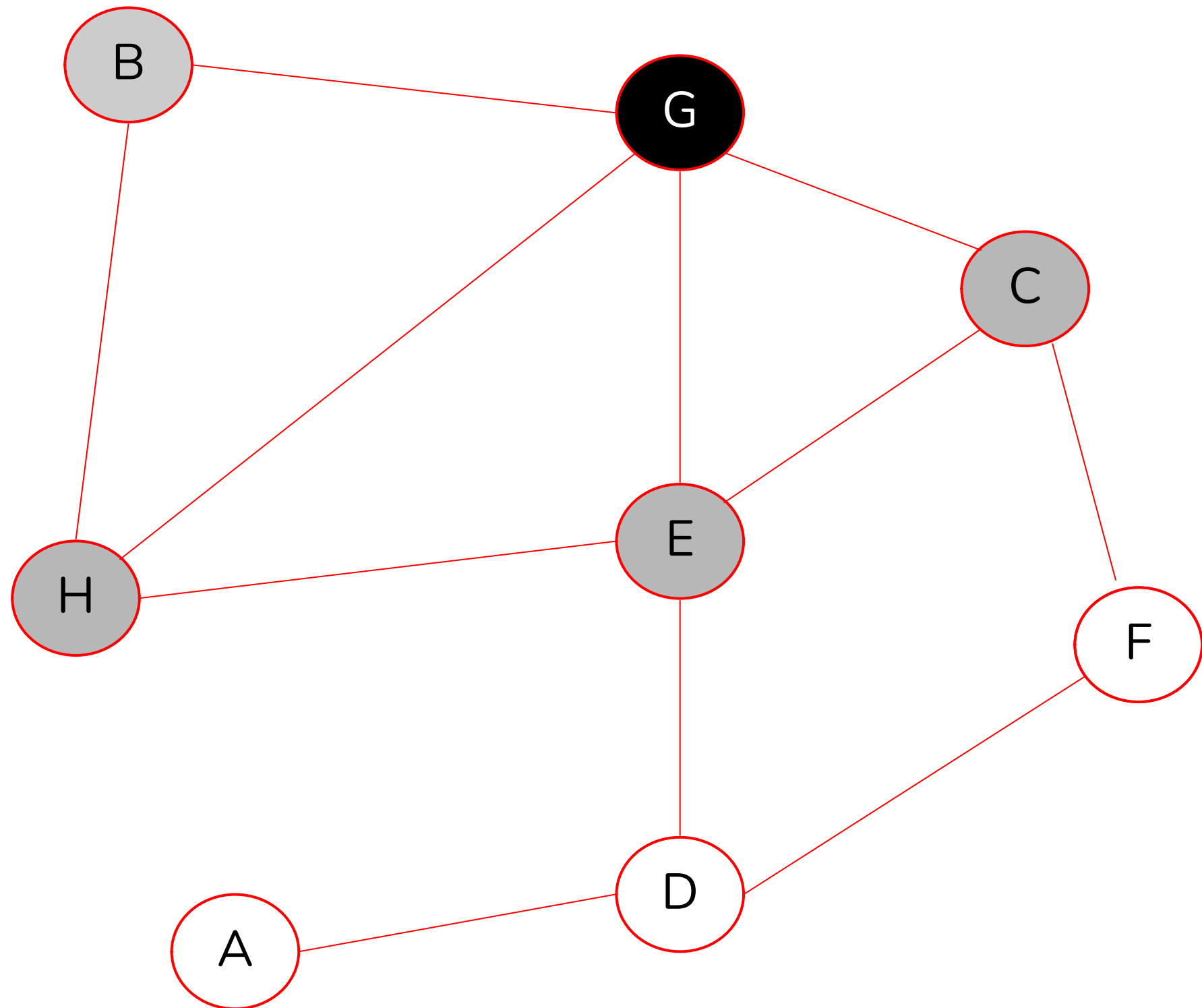


Partimos seleccionando un nodo (por ejemplo G)

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{G\}$

BFS

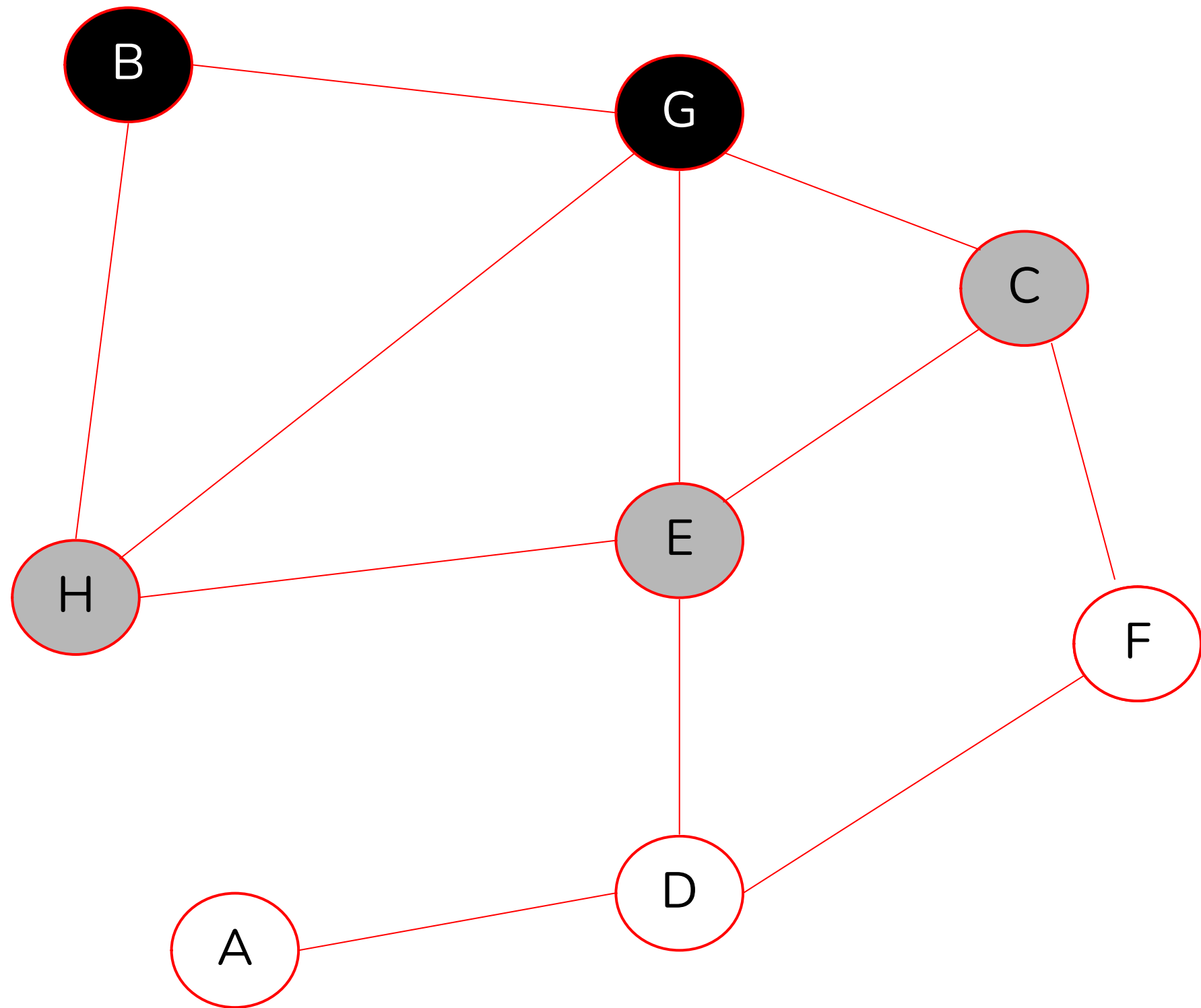


Si un nodo es gris o negro, no lo volvemos a agregar

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{B, H, E, C\}$

BFS

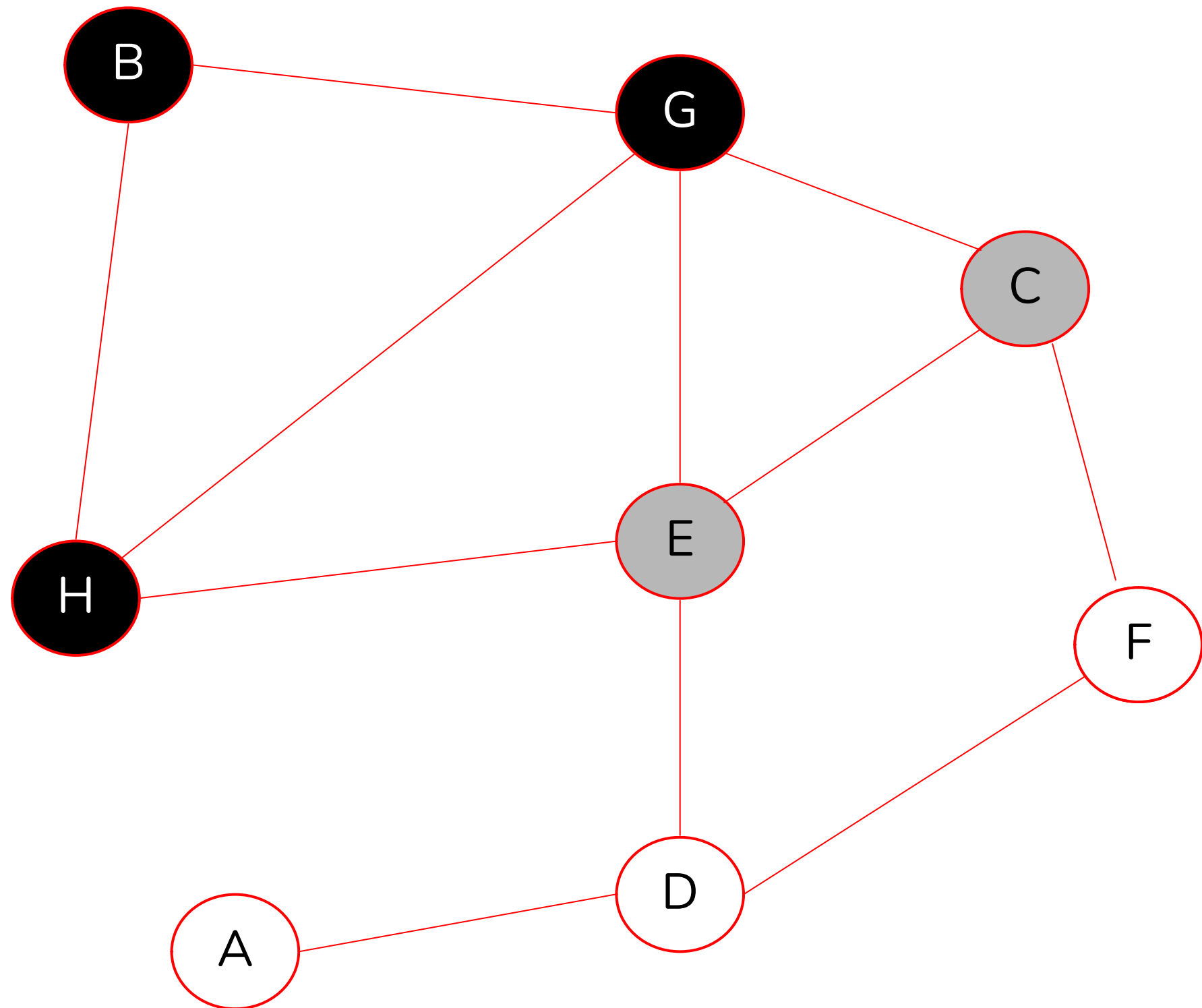


Si un nodo es gris o negro, no lo volvemos a agregar

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{H, E, C\}$

BFS

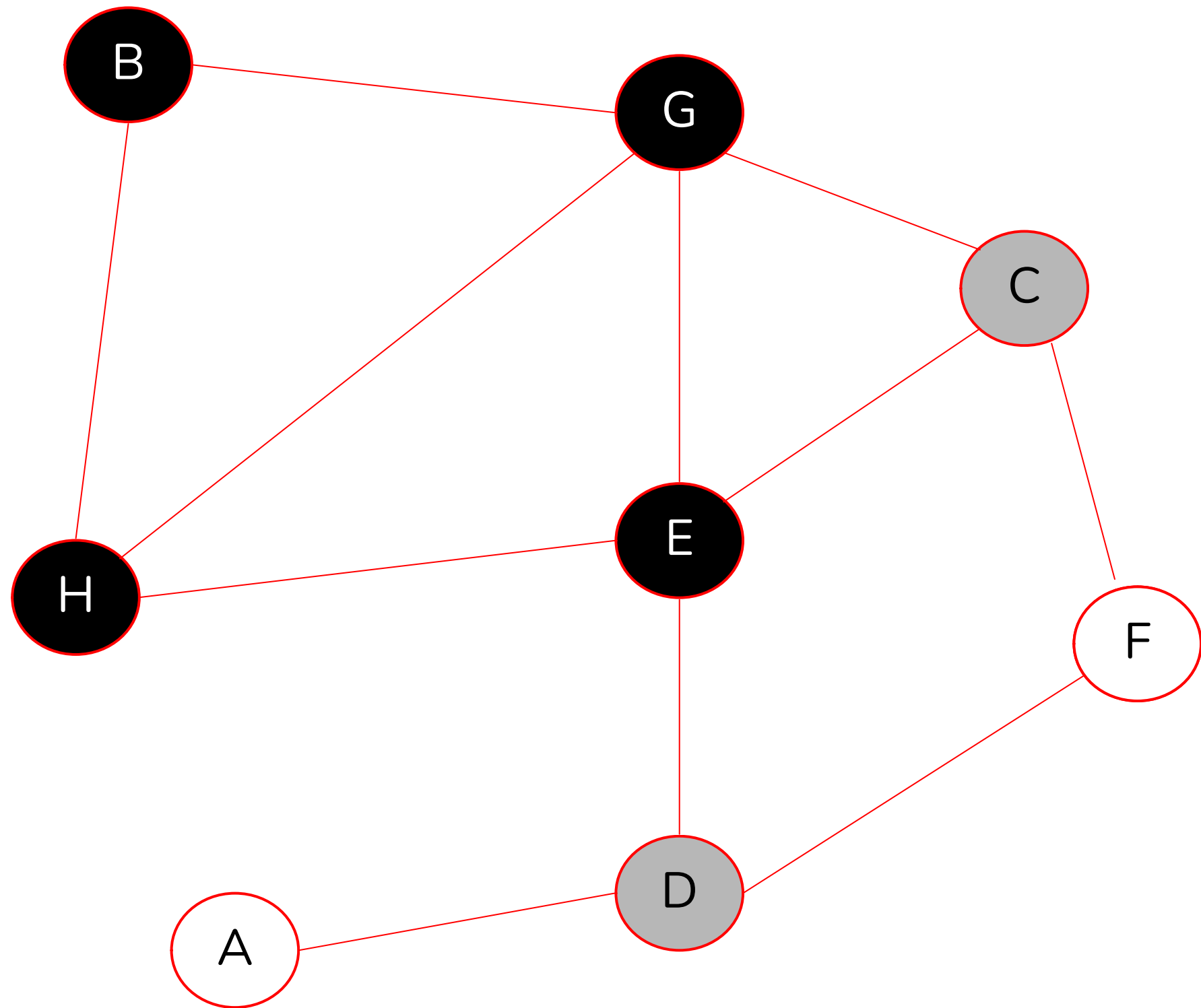


Si un nodo es gris o negro, no lo volvemos a agregar

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{E, C\}$

BFS

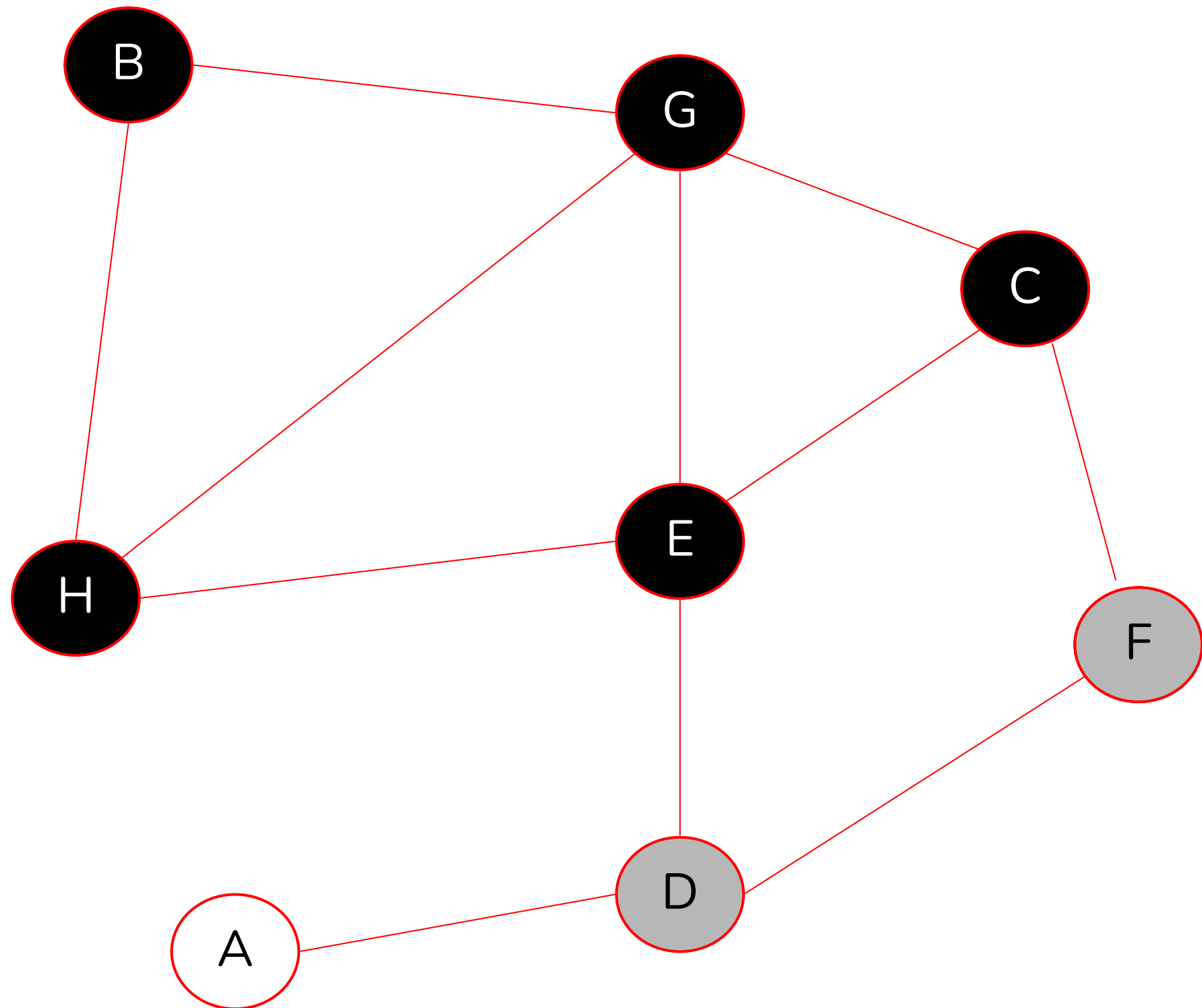


Si un nodo es gris o negro, no lo volvemos a agregar

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{C, D\}$

BFS

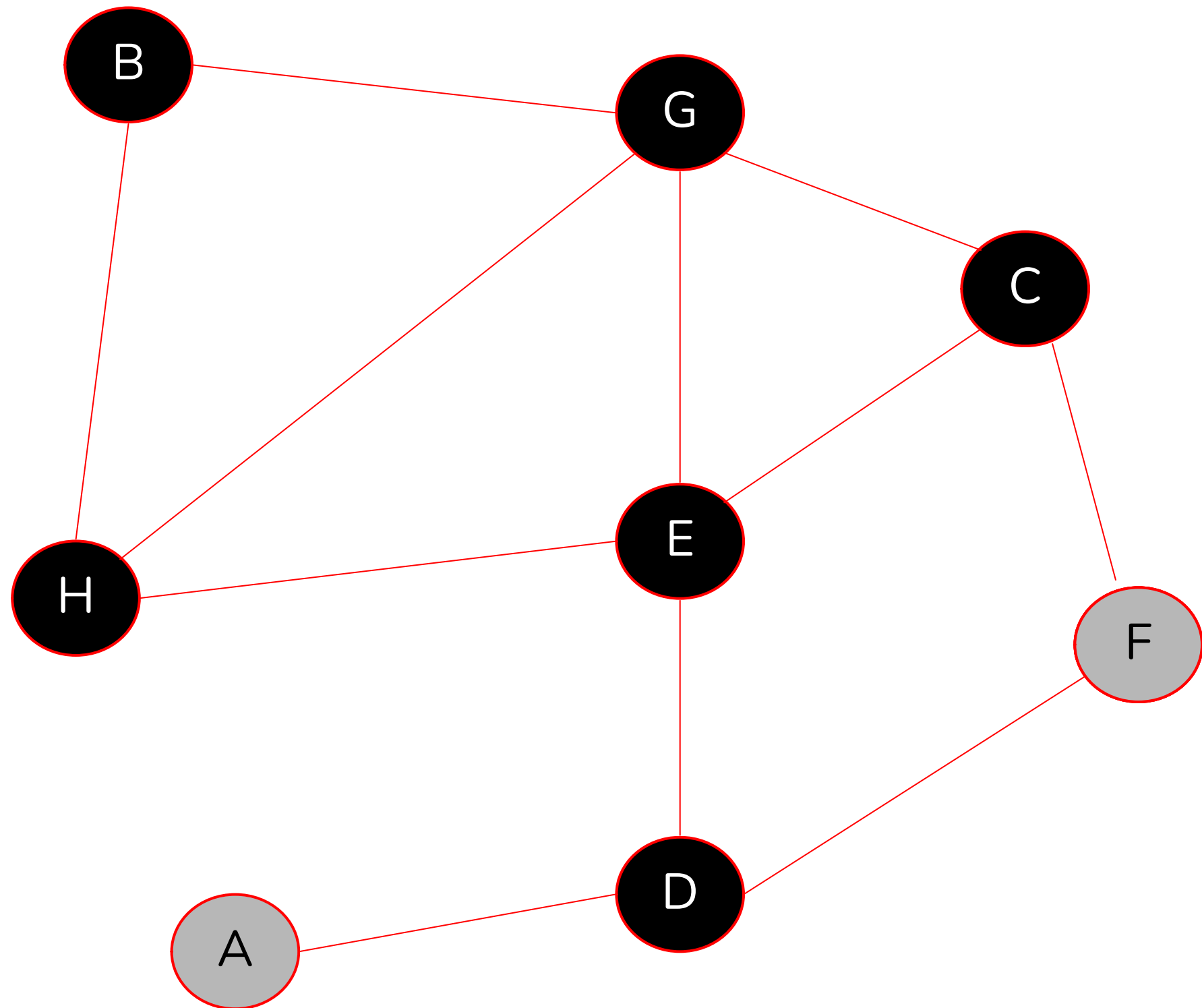


Si un nodo es gris o negro, no lo volvemos a agregar

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{D, F\}$

BFS

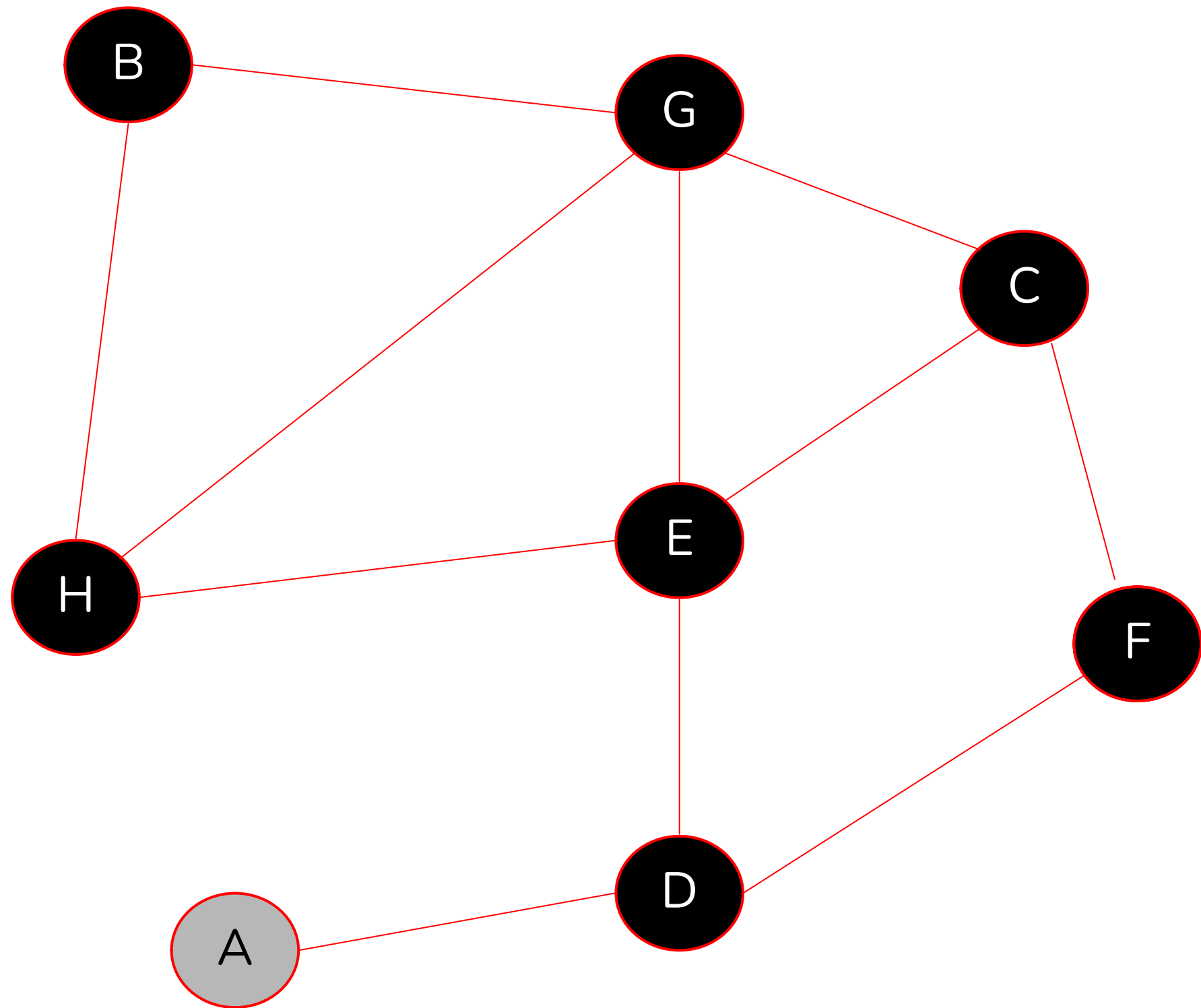


Si un nodo es gris o negro, no lo volvemos a agregar

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{F, A\}$

BFS

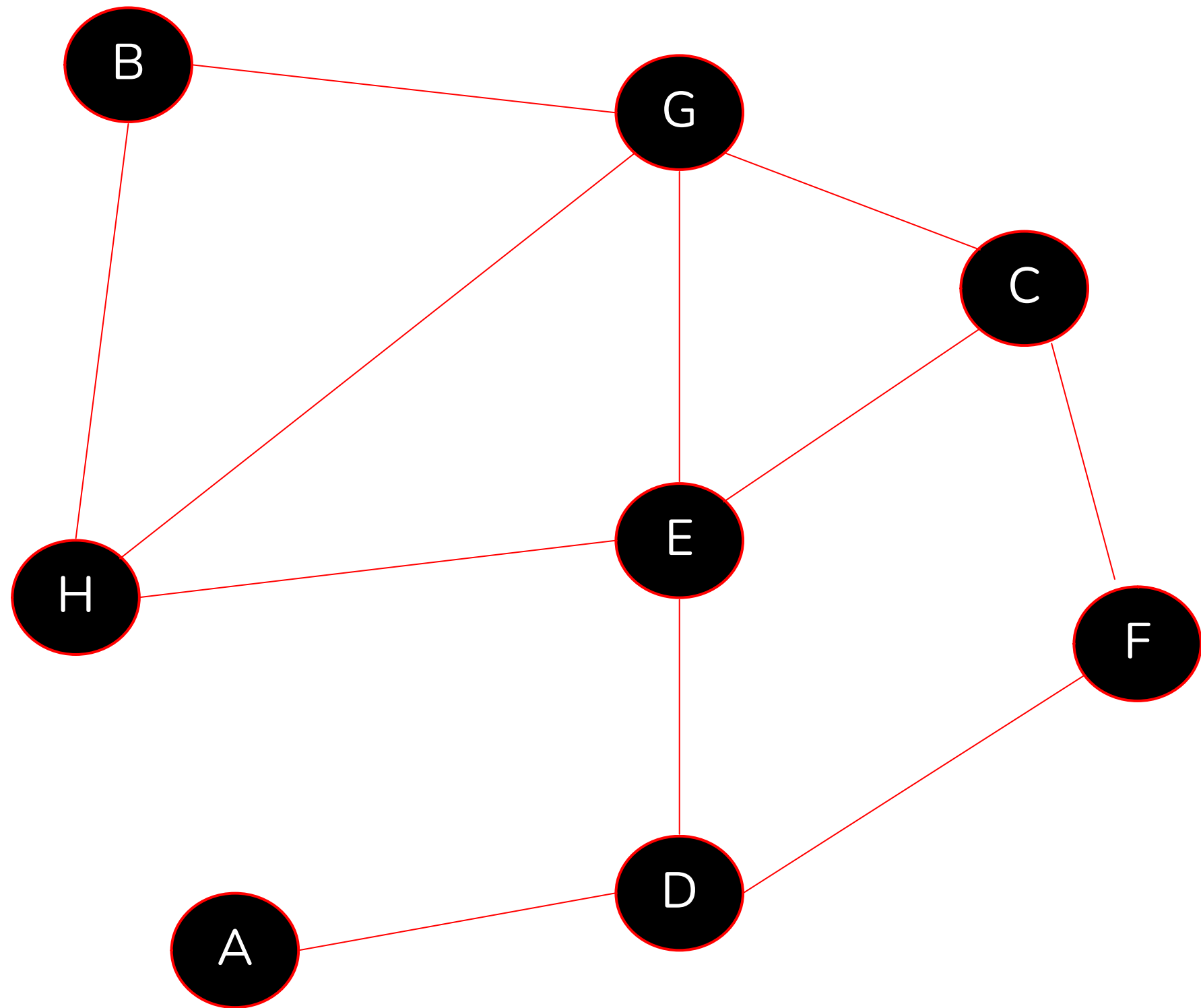


Si un nodo es gris o negro, no lo volvemos a agregar

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{A\}$

BFS



Si un nodo es gris o negro, no lo volvemos a agregar

Cada vez que agregamos un nodo a la cola lo marcamos de gris, cada vez que sacamos un nodo lo marcamos de negro

$Q = \{\}$

Una vez que la cola está vacía termina el algoritmo

EJERCICIO BFS

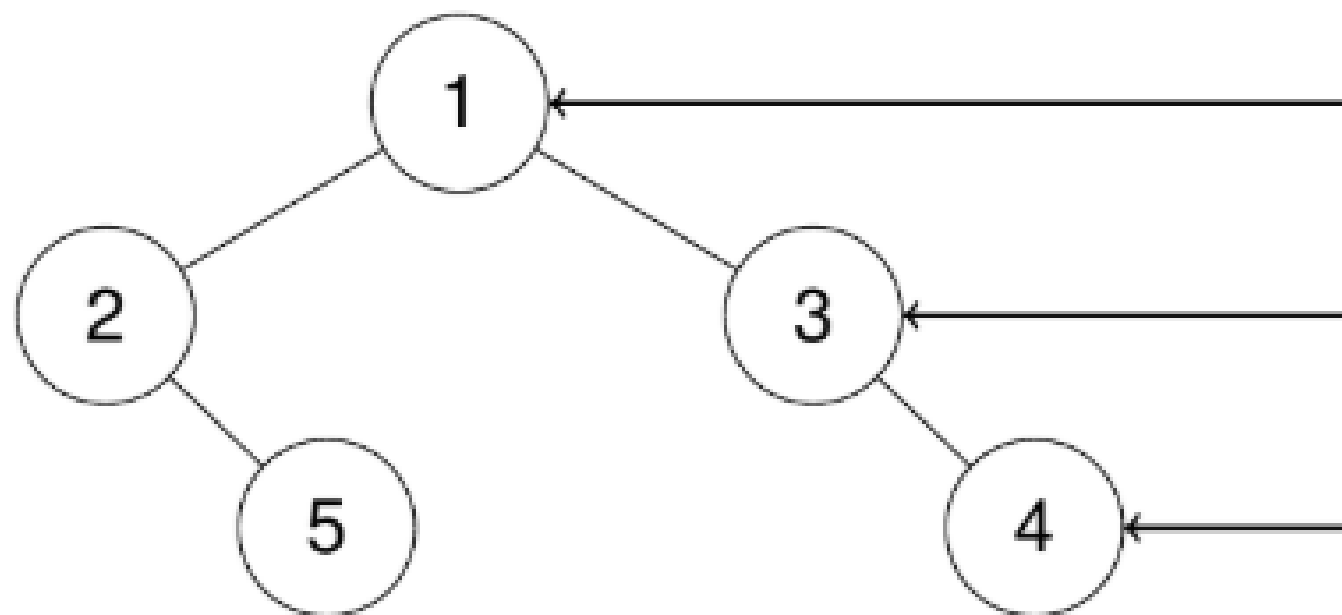
199. BINARY TREE RIGHT SIDE VIEW

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:



Dado la raíz de un árbol binario, imagínese parado en el lado derecho del árbol, Escribe el pseudocódigo *rightsideview* que recibe una raíz y retorna los valores de los nodos que puedes ver en orden de arriba hacia abajo.

199. BINARY TREE RIGHT SIDE VIEW

SOLUCION

Haz click en el link :)

FUNCIÓN rightSideView(raiz):

SI raiz ES nulo:

RETORNAR lista vacía

cola ← [raiz]

vista ← lista vacía

MIENTRAS cola NO esté vacía:

tamaño ← longitud de cola

PARA i DESDE 0 HASTA tamaño - 1:

nodoActual ← eliminar_primer_elemento(cola)

SI nodoActual.tiene_izquierdo:

agregar nodoActual.izquierdo a cola

SI nodoActual.tiene_derecho:

agregar nodoActual.derecho a cola

// al finalizar el nivel, nodoActual es el último del nivel

agregar nodoActual.valor a vista

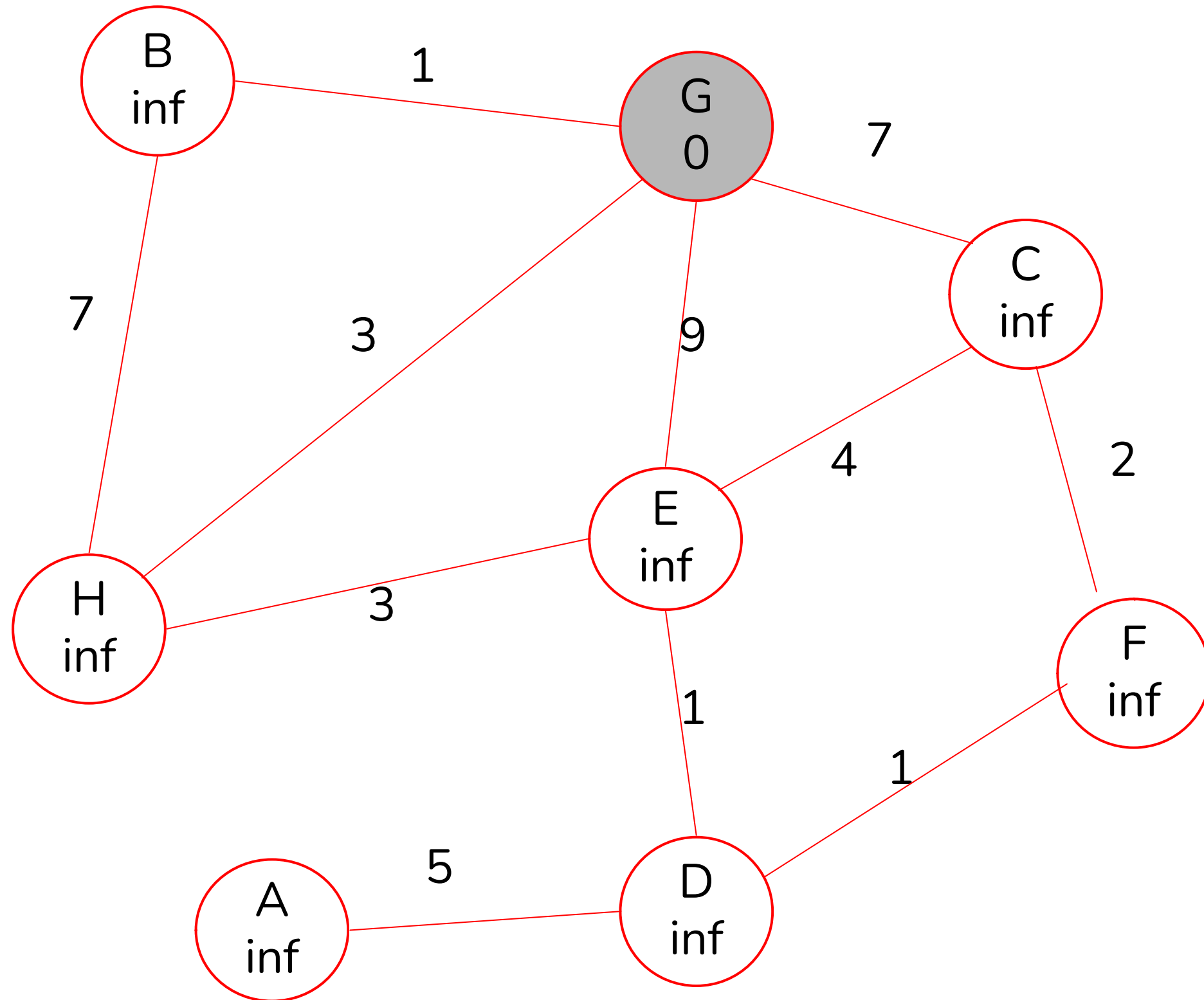
RETORNAR vista

DIJKSTRA'S ALGORITHM

¿CÓMO PODEMOS MEJORAR BFS PARA ENCONTRAR LA RUTA MÁS CORTA?

- **Cambiamos la cola por un min-heap**, que ordene las aristas según peso de menor a mayor, de tal forma que siempre revisemos primero posibles caminos más cortos.
- Iniciamos todos los nodos con distancia infinita a nuestro nodo inicial, de tal forma que cada vez que lo visitamos revisamos si encontramos una distancia menor

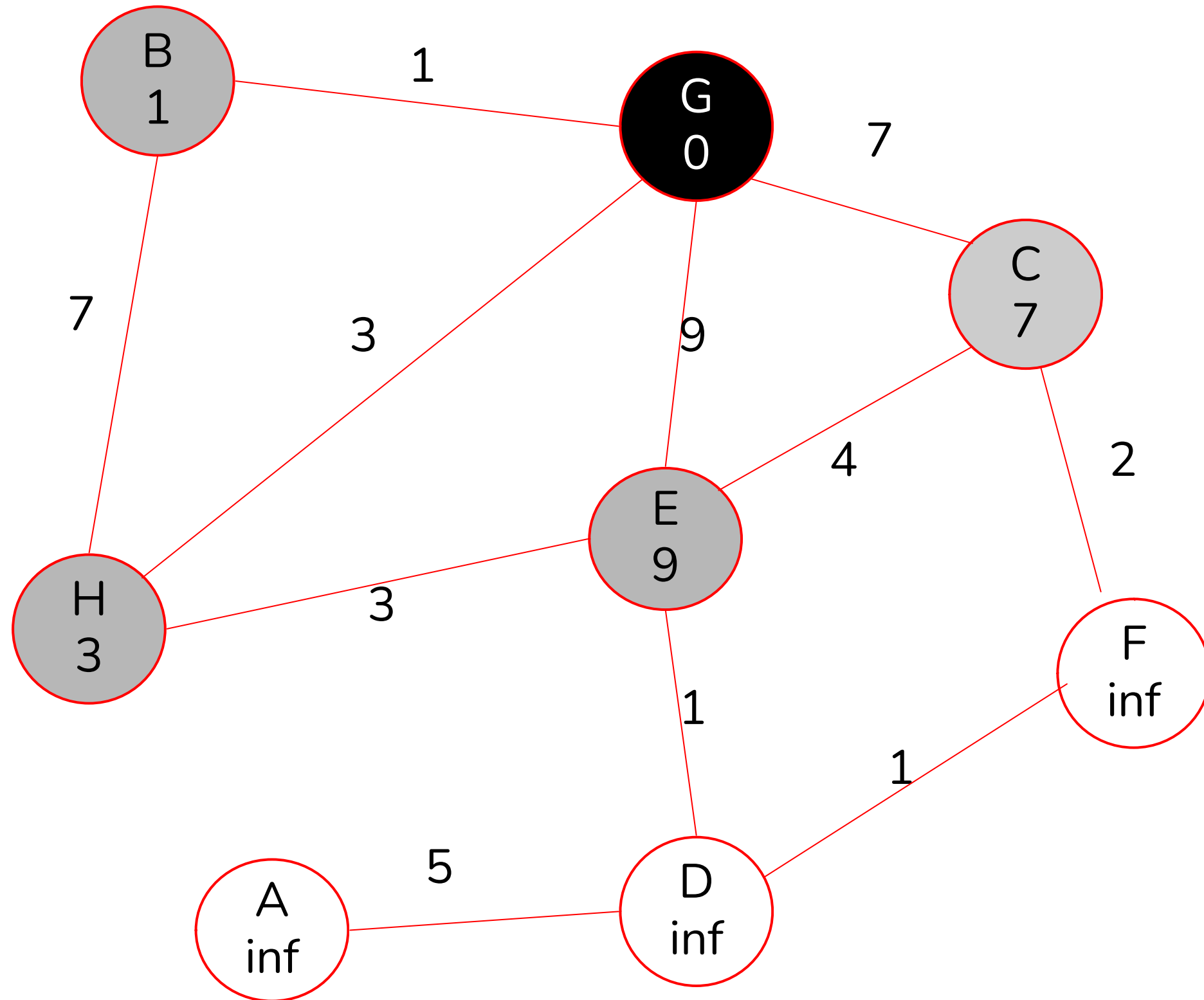
DIJKSTRA



Partimos desde el nodo G, similar a BFS
marcamos los nodos con los mismos códigos
de color (PQ por Priority Queue)

$PQ = \{(G, 0)\}$

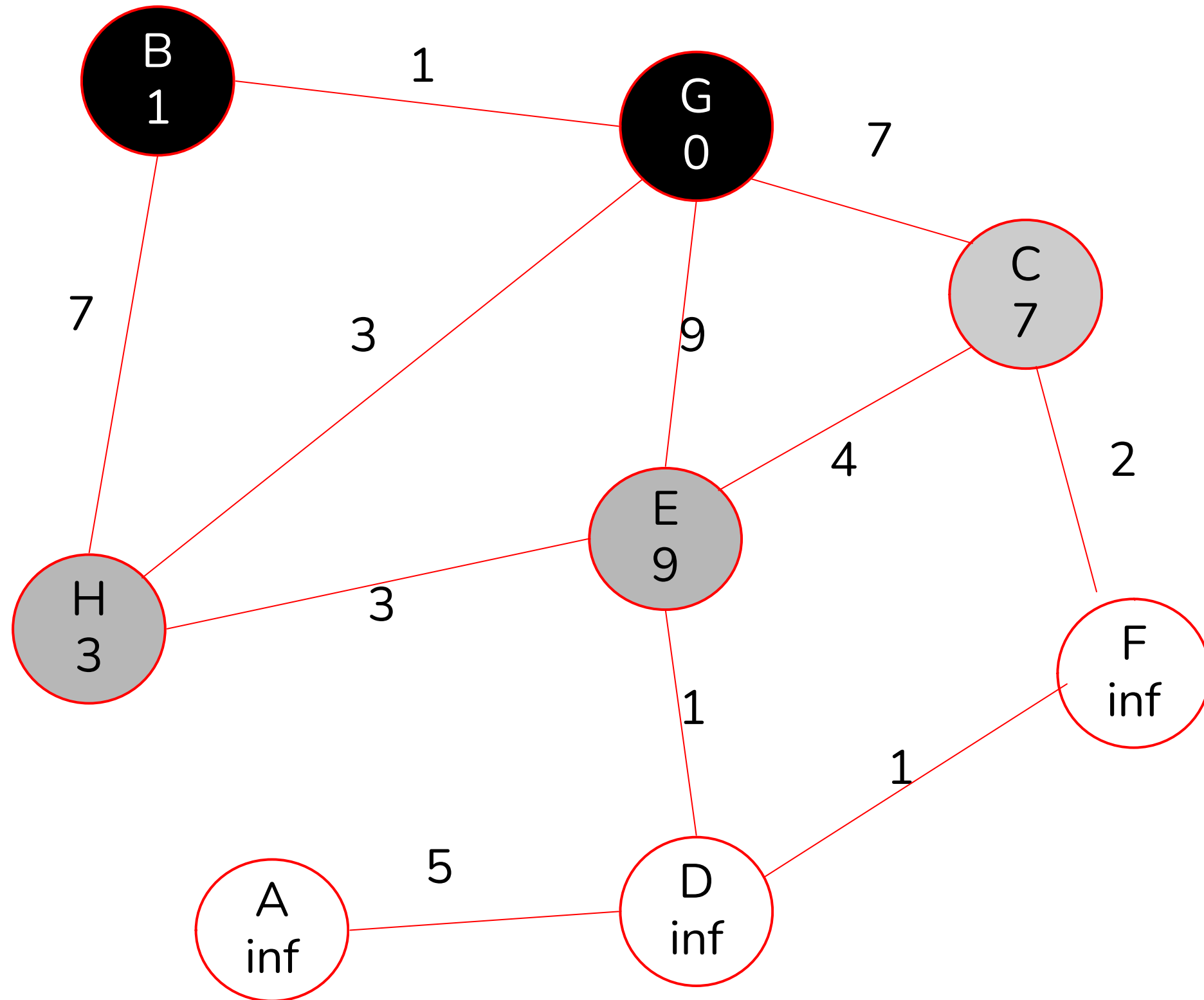
DIJKSTRA



Para calcular las distancias, sumamos el peso de la arista con la distancia que lleva acumulada el nodo

$PQ = \{(B, 1), (H, 3), (C, 7), (E, 9)\}$

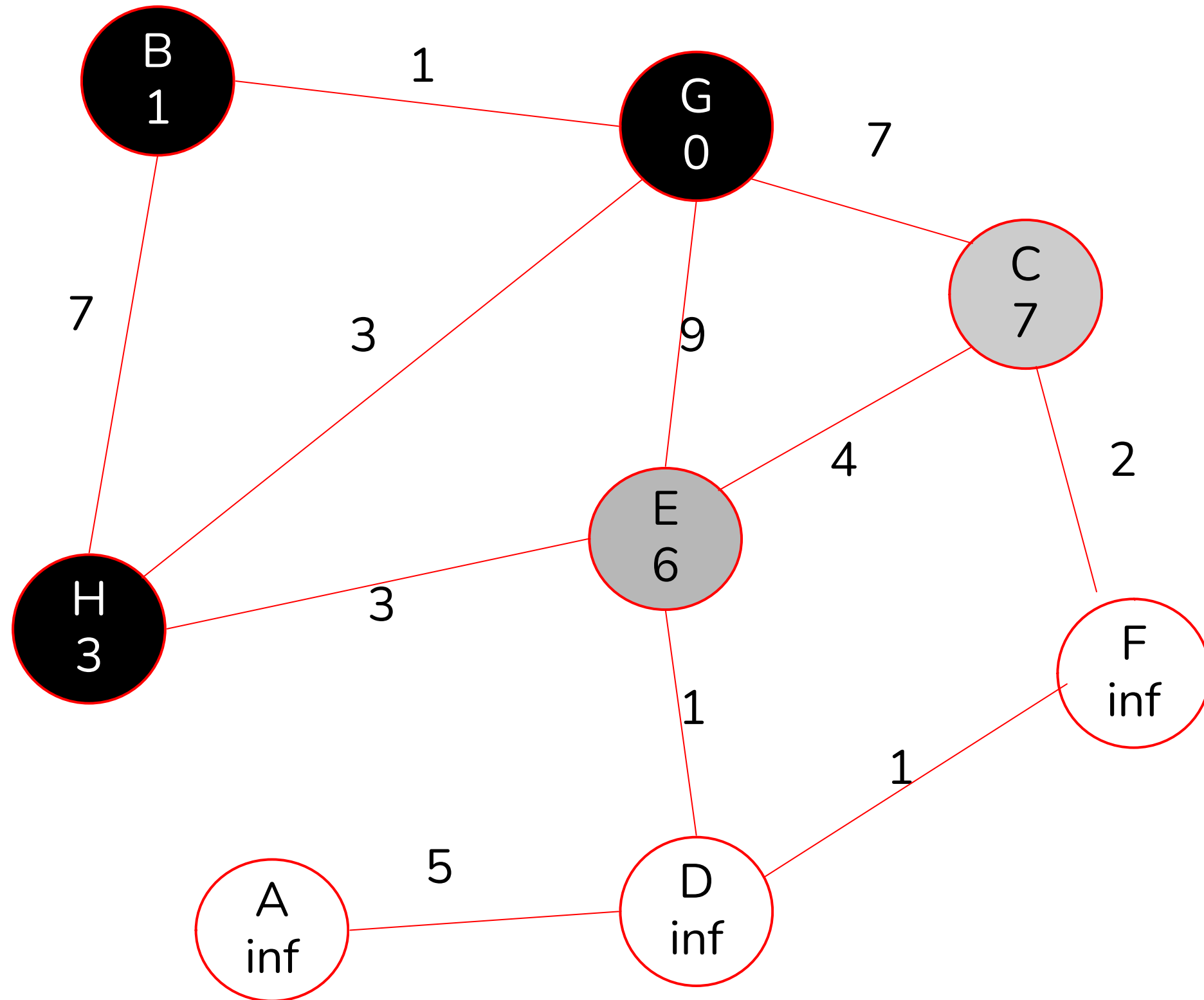
DIJKSTRA



Para este caso tenemos que la distancia de G a H pasando por B es mayor que la que teníamos

$PQ = \{(H, 3), (C, 7), (E, 9)\}$

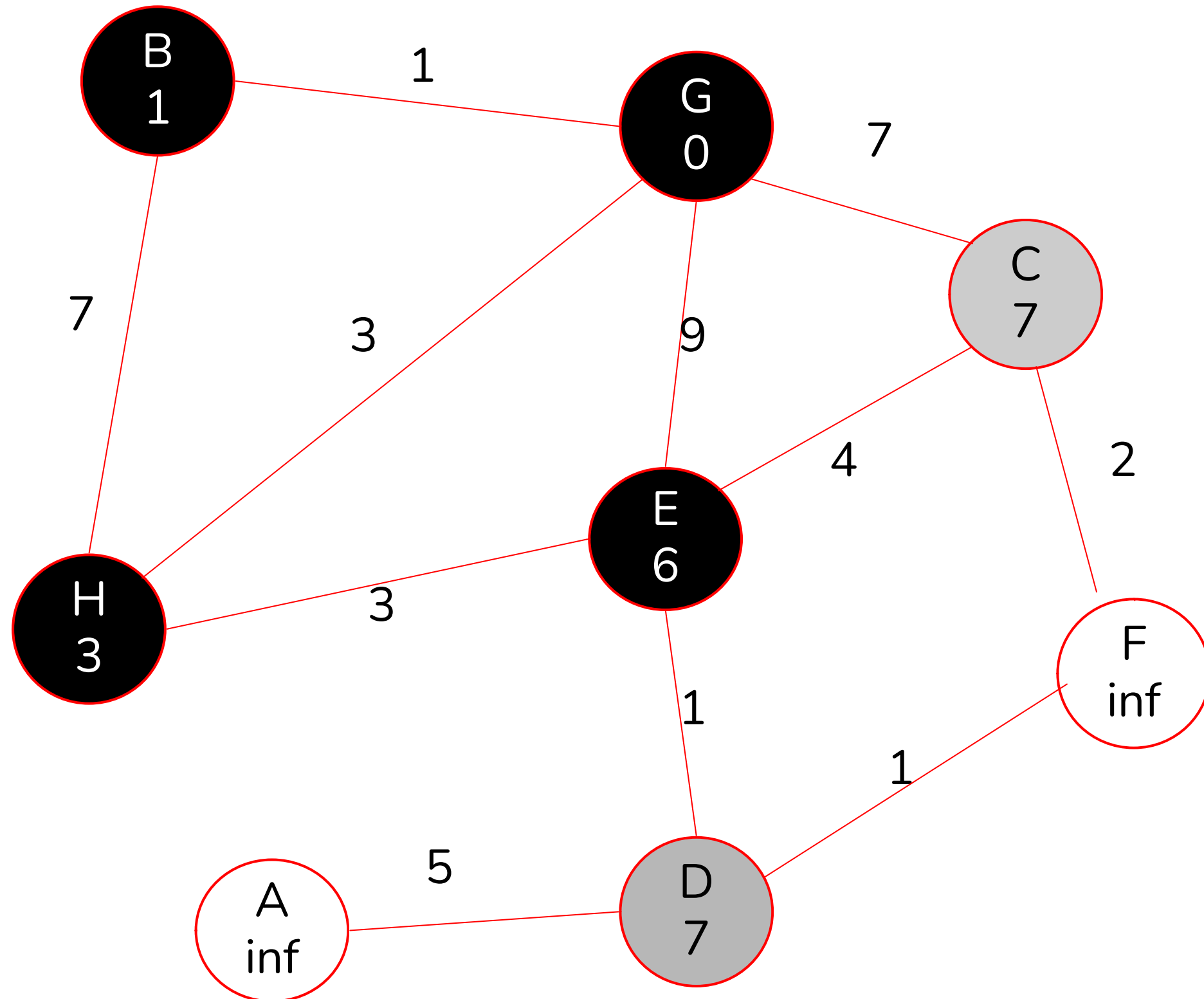
DIJKSTRA



Para este caso tenemos que la distancia de G a E pasando por H es menor que la que teníamos

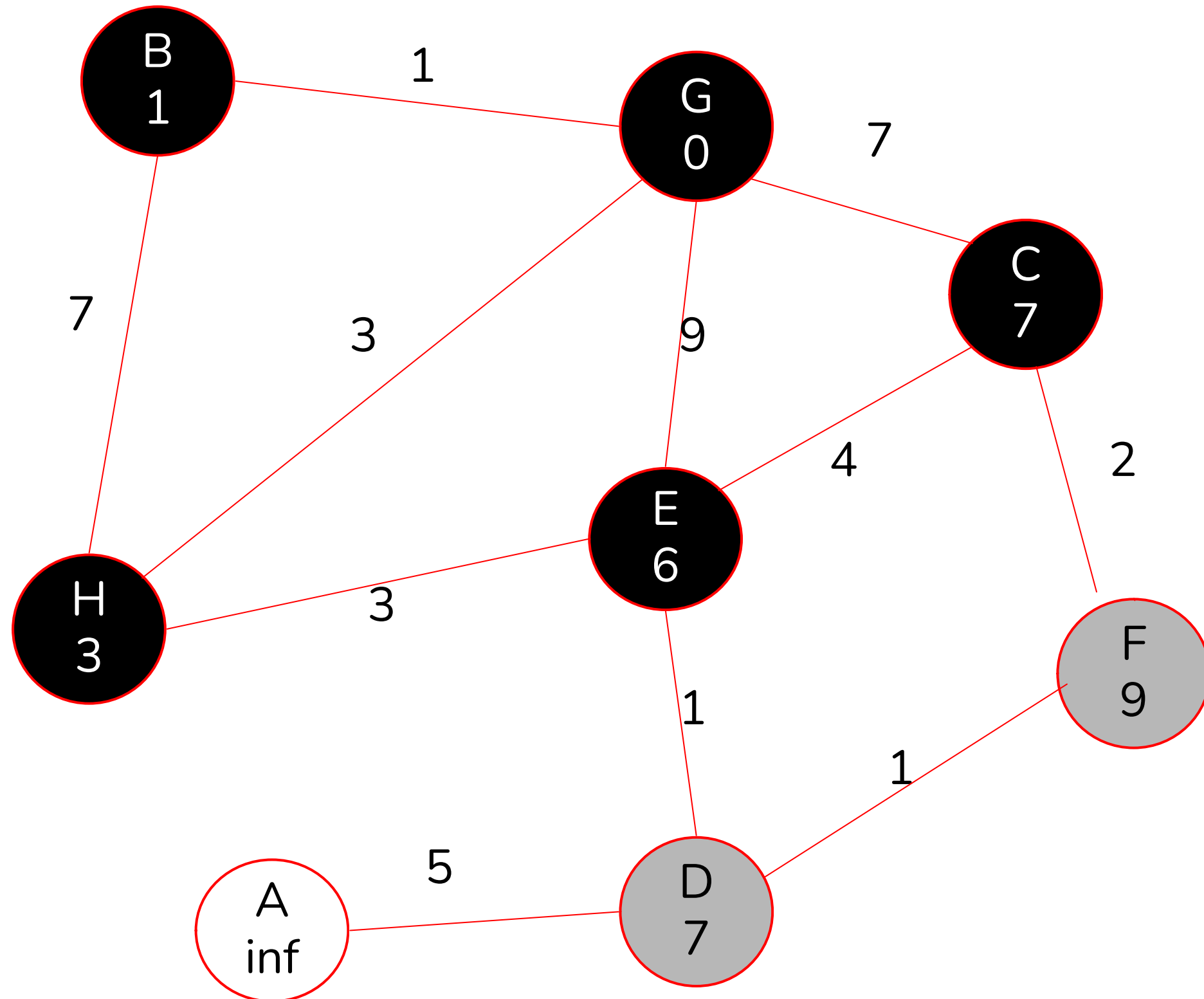
$PQ = \{(E, 6), (C, 7)\}$

DIJKSTRA



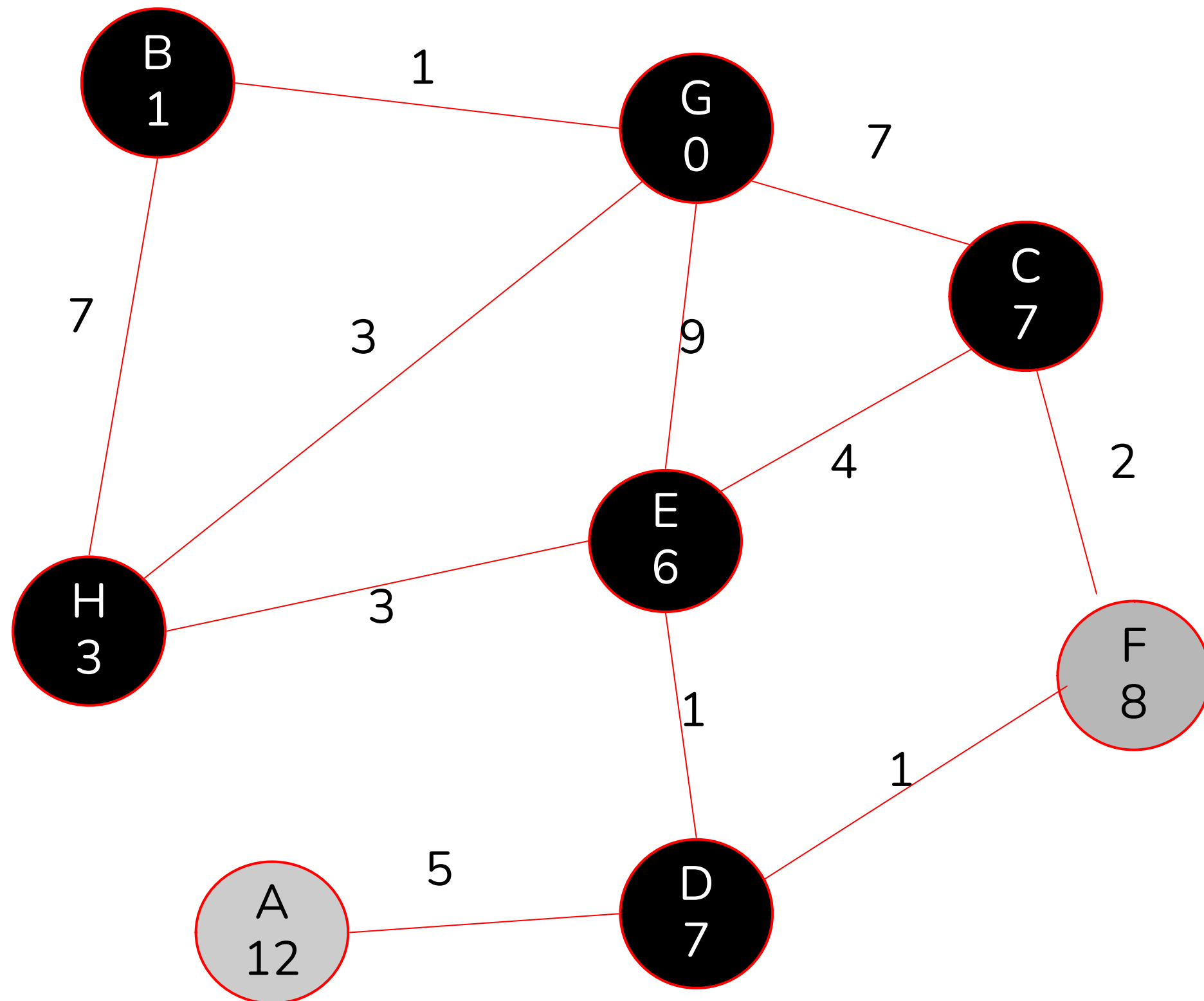
PQ = {(C, 7), (D, 7)}

DIJKSTRA



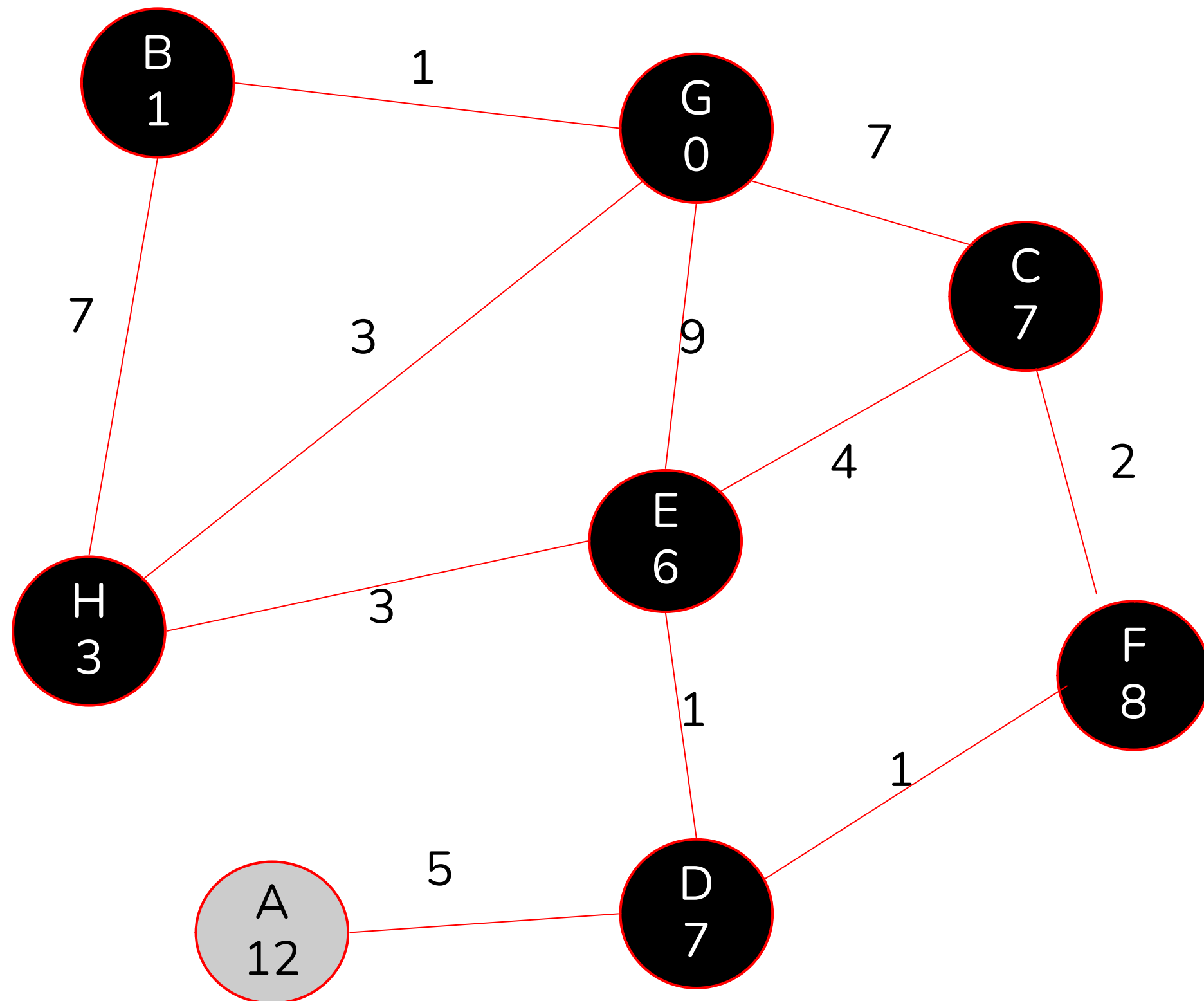
PQ = {(D, 7), (F, 9)}

DIJKSTRA



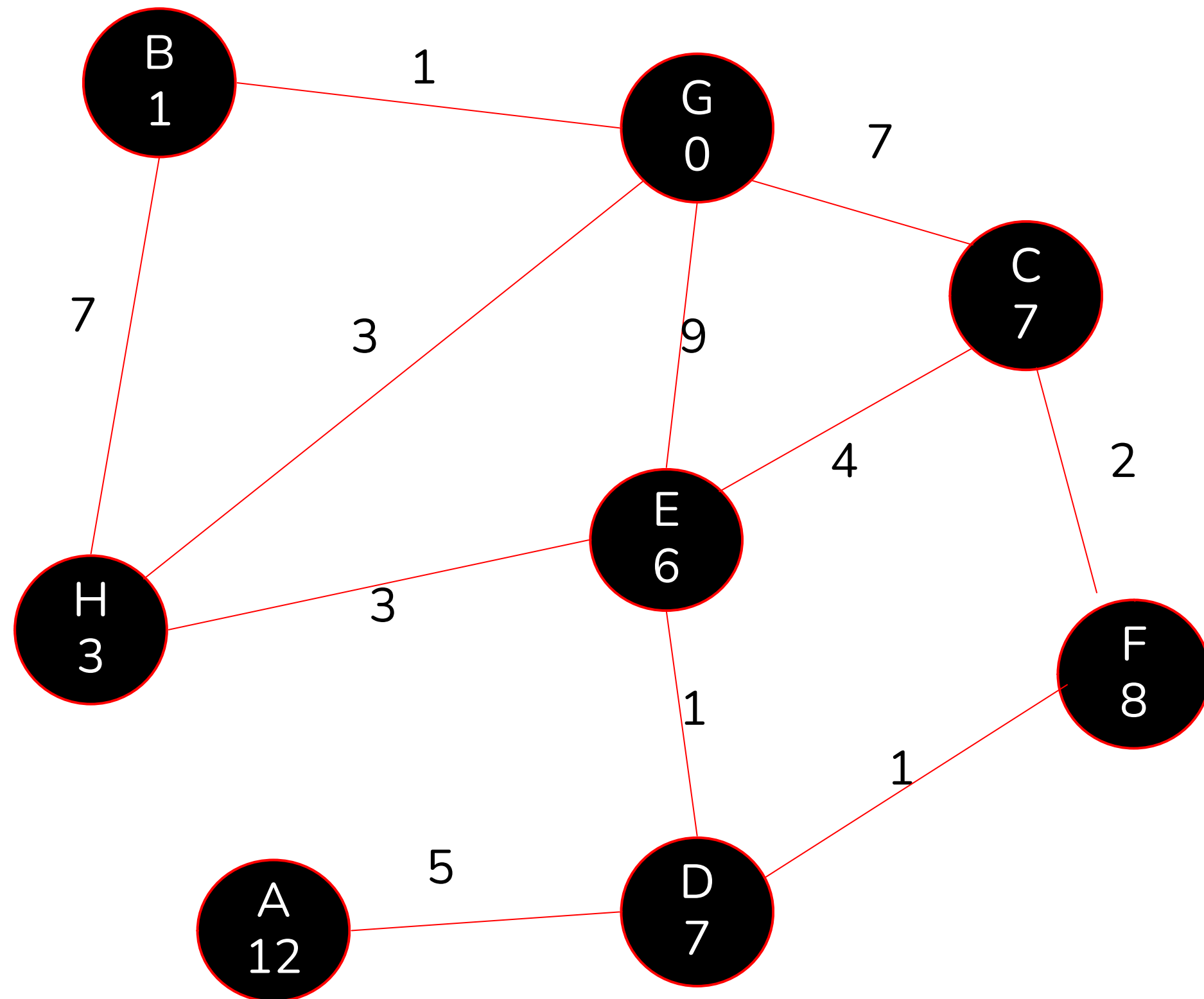
PQ = {(F, 8), (A, 12)}

DIJKSTRA



PQ = {(A, 12)}

DIJKSTRA



PQ = {}

EJERCICIO DIJKSTRA

I3 2022-2 P4

- (b) [2 ptos.] El diámetro de un árbol no dirigido conexo T se define como el largo del camino más largo en T . Suponga que existe un único camino de largo máximo en T con extremos u y v . Si x es un nodo cualquiera, se puede demostrar que el nodo más lejano a x es u o v . Usando este resultado, proponga un algoritmo que determine el diámetro de un árbol T .

I3 2022-2 P4

- (b) [2 ptos.] El diámetro de un árbol no dirigido conexo T se define como el largo del camino más largo en T . Suponga que existe un único camino de largo máximo en T con extremos u y v . Si x es un nodo cualquiera, se puede demostrar que el nodo más lejano a x es u o v . Usando este resultado, proponga un algoritmo que determine el diámetro de un árbol T .

La propiedad descrita permite plantear el siguiente algoritmo

Diameter(V, E):

```
1   $x \leftarrow$  cualquier nodo de  $V$ 
2   $d \leftarrow$  BFS( $x$ )
3   $u \leftarrow$  nodo que tiene máximo  $d[\cdot]$ 
4   $d \leftarrow$  BFS( $u$ )
5   $D \leftarrow \max\{d[v] \mid v \in V\}$ 
6  return  $D$ 
```

donde se usa una versión modificada de BFS para que retorne el arreglo con distancias desde la fuente, así como el nodo asociado a cada distancia.

BELLMAN-FORD

BELLMAN-FORD

A diferencia de los dos otros algoritmos vistos, Bellman-Ford permite resolver el problema de encontrar los caminos más cortos pero para grafos que permiten tener aristas con pesos negativos.

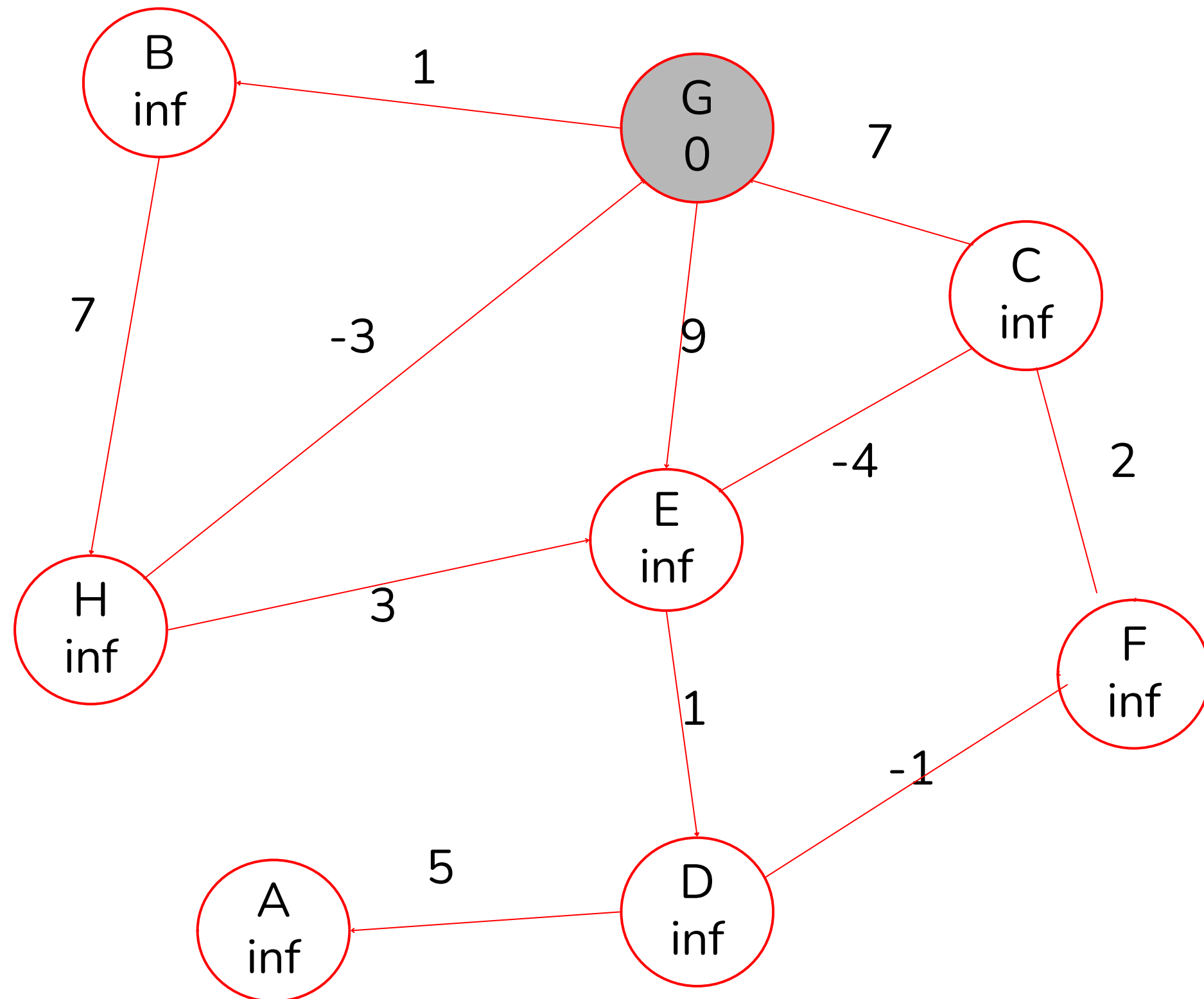
- Este algoritmo funciona con grafos dirigidos y aristas con pesos
- Detecta ciclos negativos

BELLMAN-FORD

```
BellmanFord(s):  
1  for  $u \in V$  :  
2       $d[u] \leftarrow \infty$ ;  $\pi[u] \leftarrow \emptyset$   
3   $d[s] \leftarrow 0$   
4  for  $k = 1 \dots |V| - 1$  :  
5      for  $(u, v) \in E$  :  
6          if  $d[v] > d[u] + \text{cost}(u, v)$  :  
7               $d[v] \leftarrow d[u] + \text{cost}(u, v)$   
8               $\pi[v] \leftarrow u$ 
```

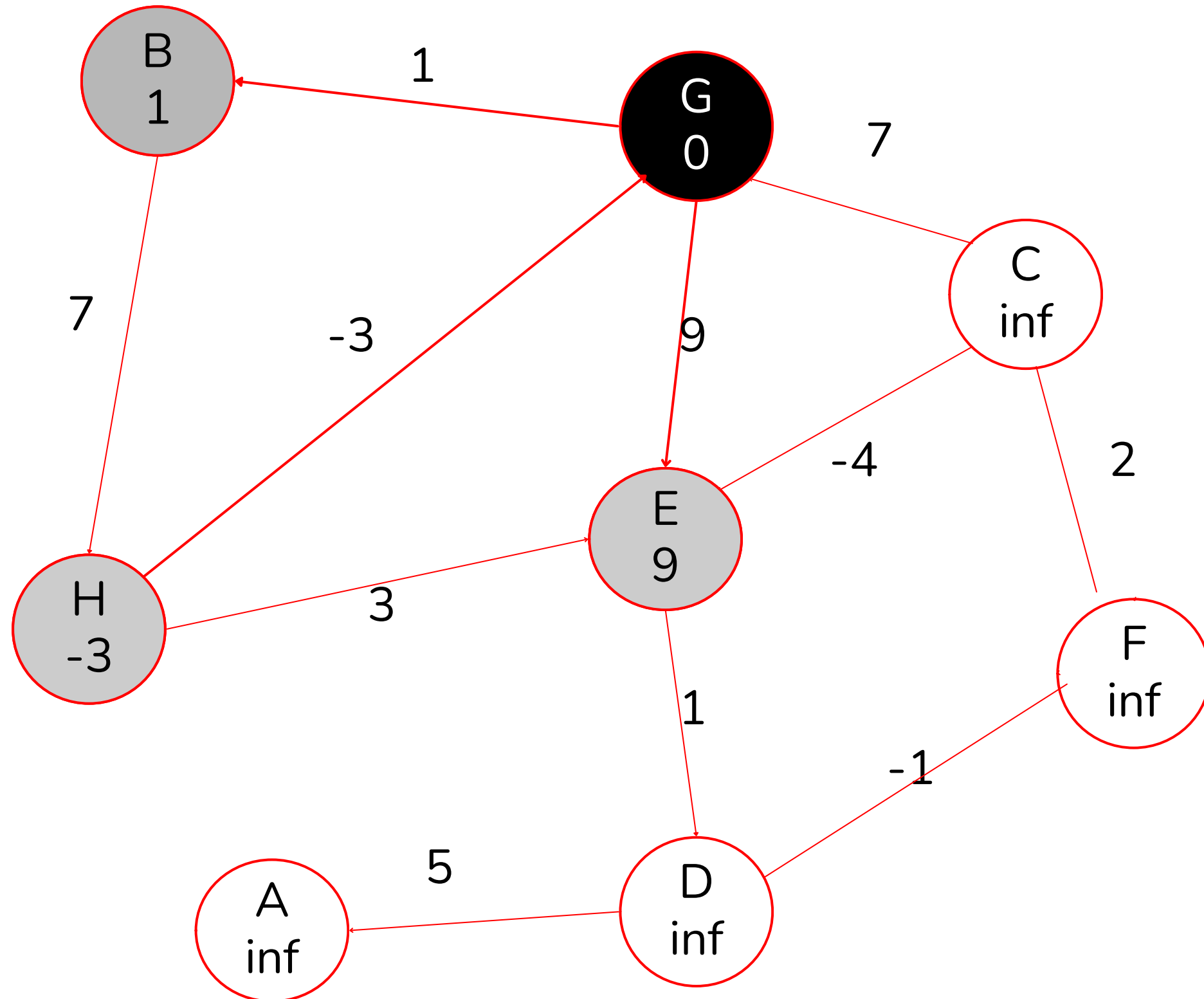
```
ValidBellmanFord(s):  
1  for  $u \in V$  :  
2       $d[u] \leftarrow \infty$ ;  $\pi[u] \leftarrow \emptyset$   
3   $d[s] \leftarrow 0$   
4  for  $k = 1 \dots |V| - 1$  :  
5      for  $(u, v) \in E$  :  
6          if  $d[v] > d[u] + \text{cost}(u, v)$  :  
7               $d[v] \leftarrow d[u] + \text{cost}(u, v)$   
8               $\pi[v] \leftarrow u$   
9  for  $(u, v) \in E$  :  
10     if  $d[v] > d[u] + \text{cost}(u, v)$  :  
11         return false  
12 return true
```

BELLMAN-FORD

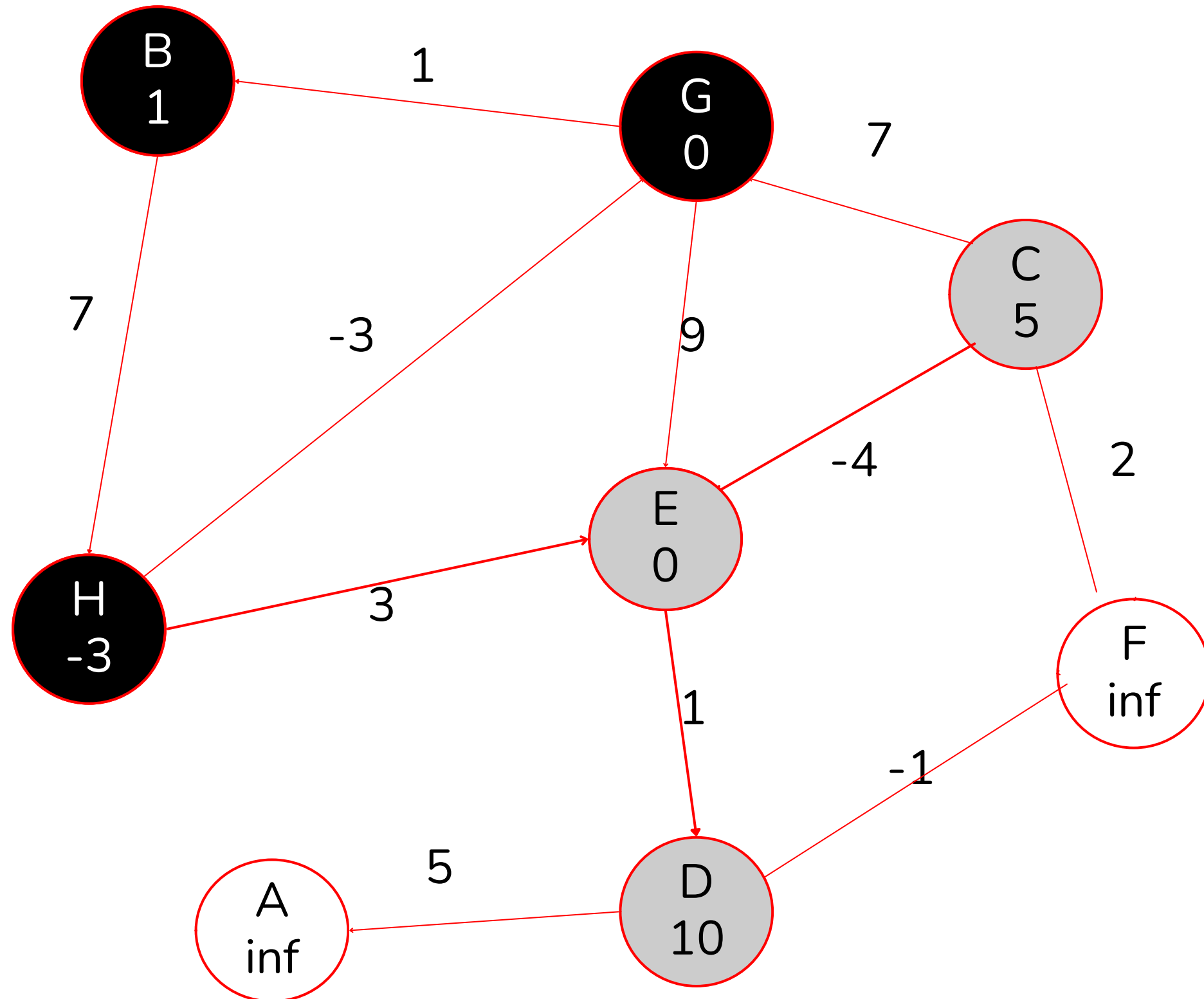


Notemos que ahora tenemos grafos dirigidos

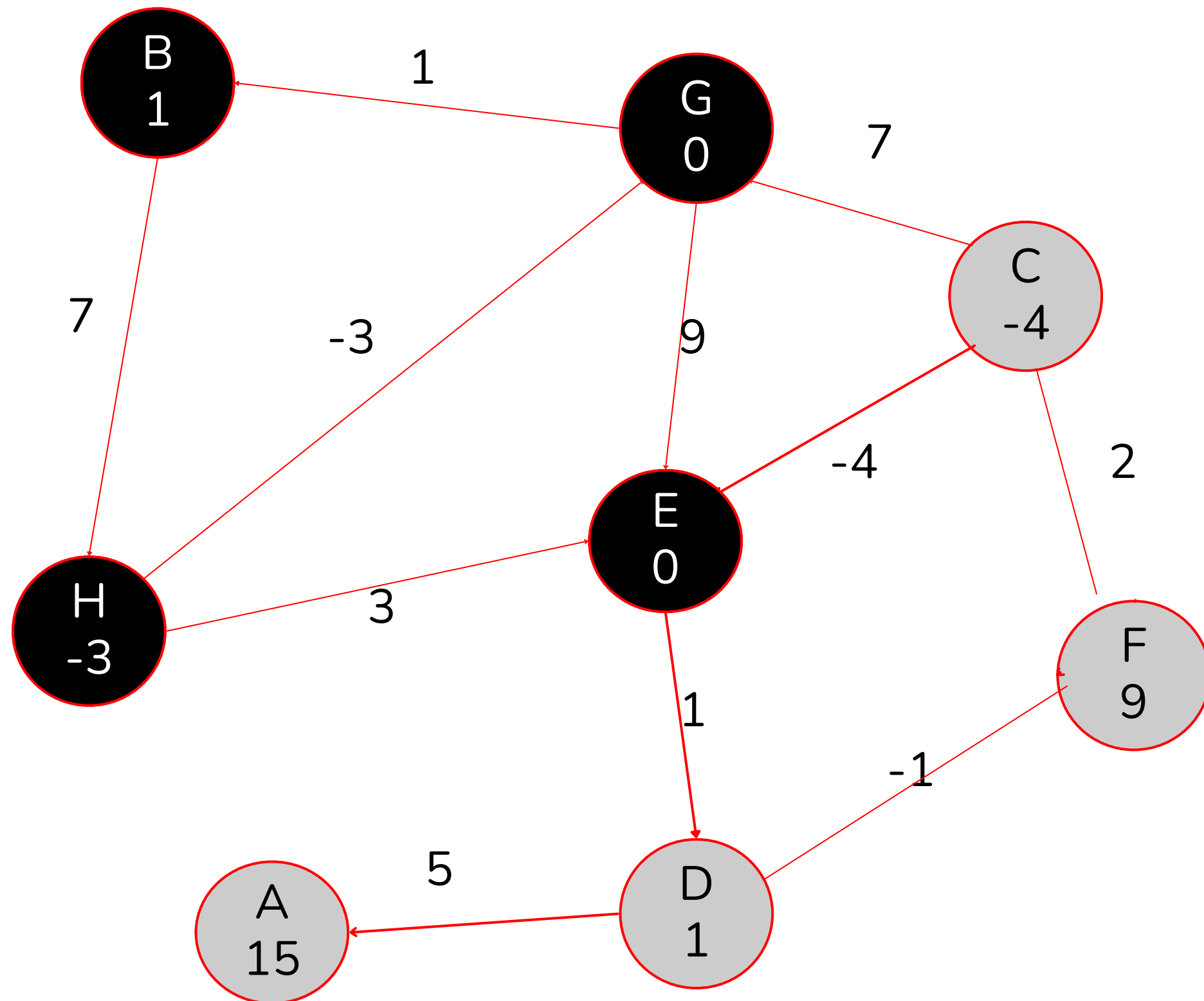
BELLMAN-FORD



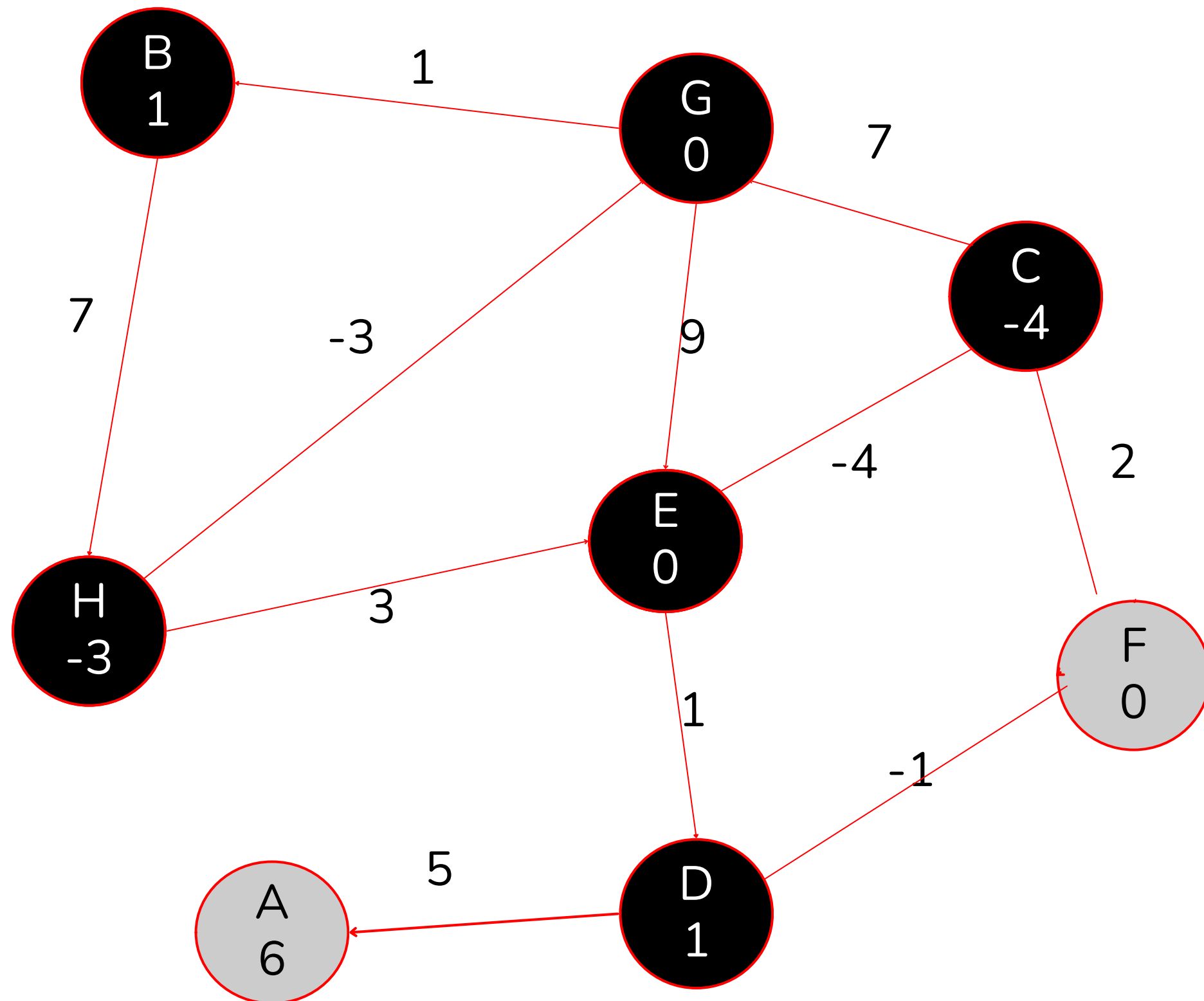
BELLMAN-FORD



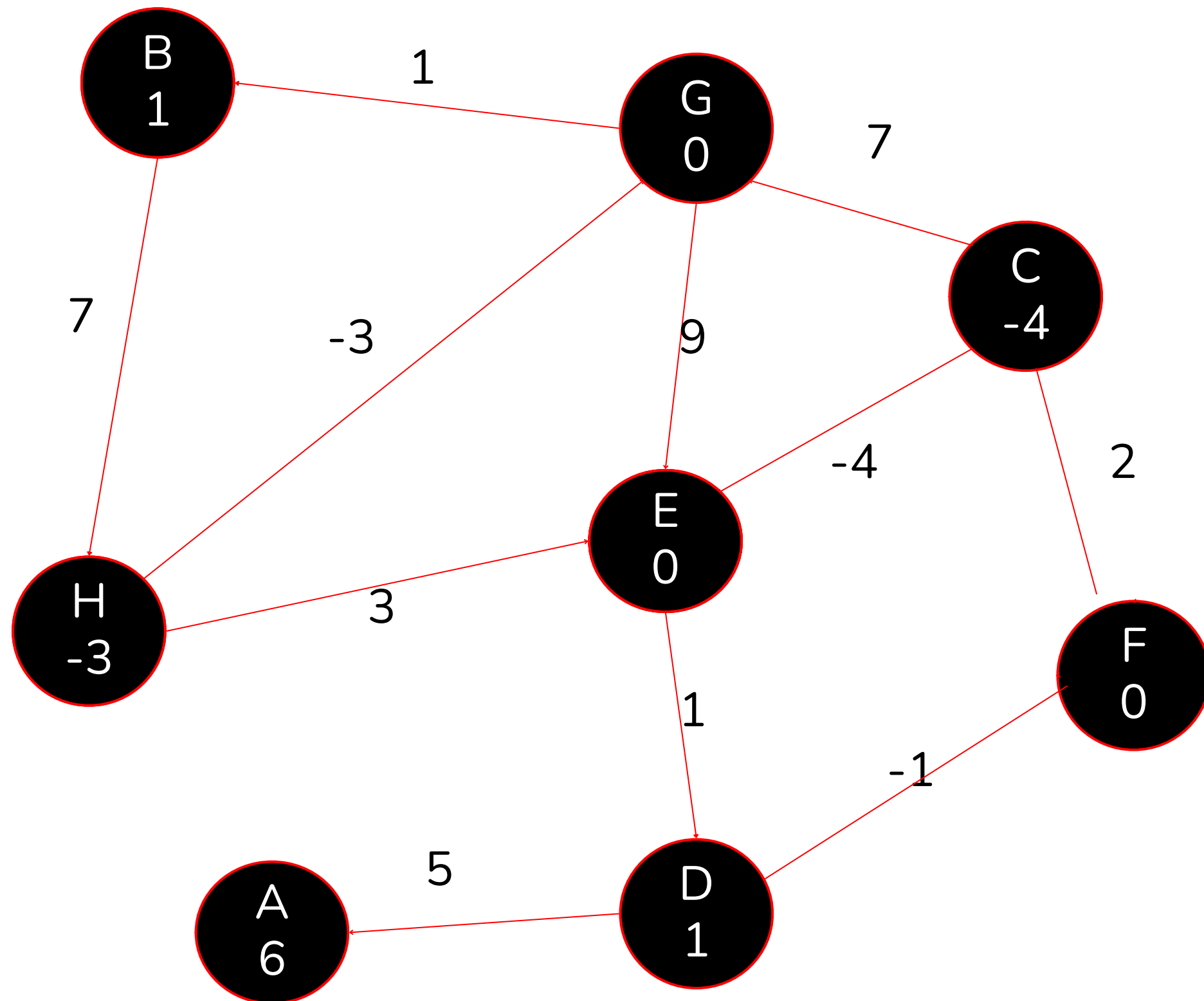
BELLMAN-FORD



BELLMAN-FORD



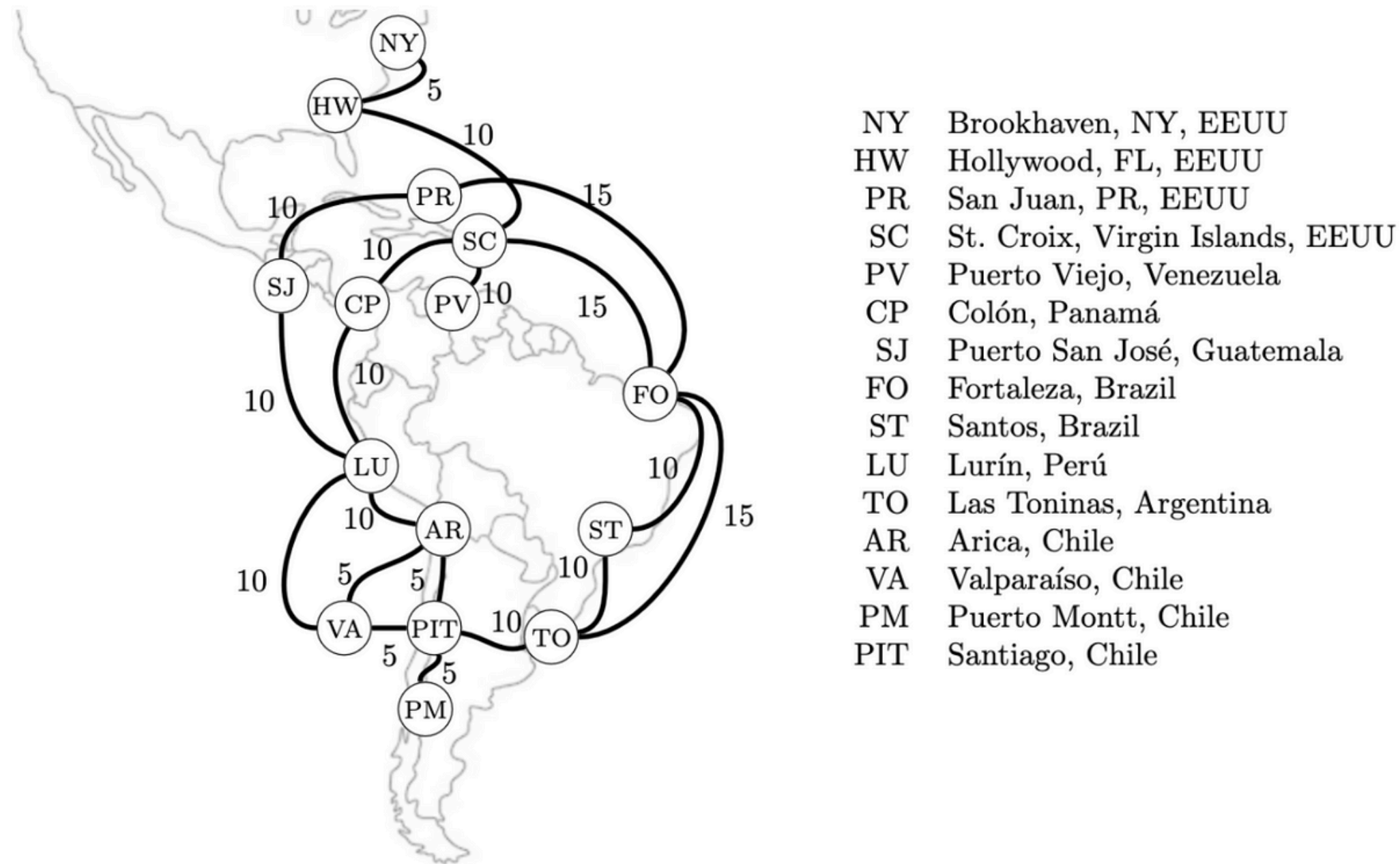
BELLMAN-FORD



EJERCICIOS BELLMAN-FORD

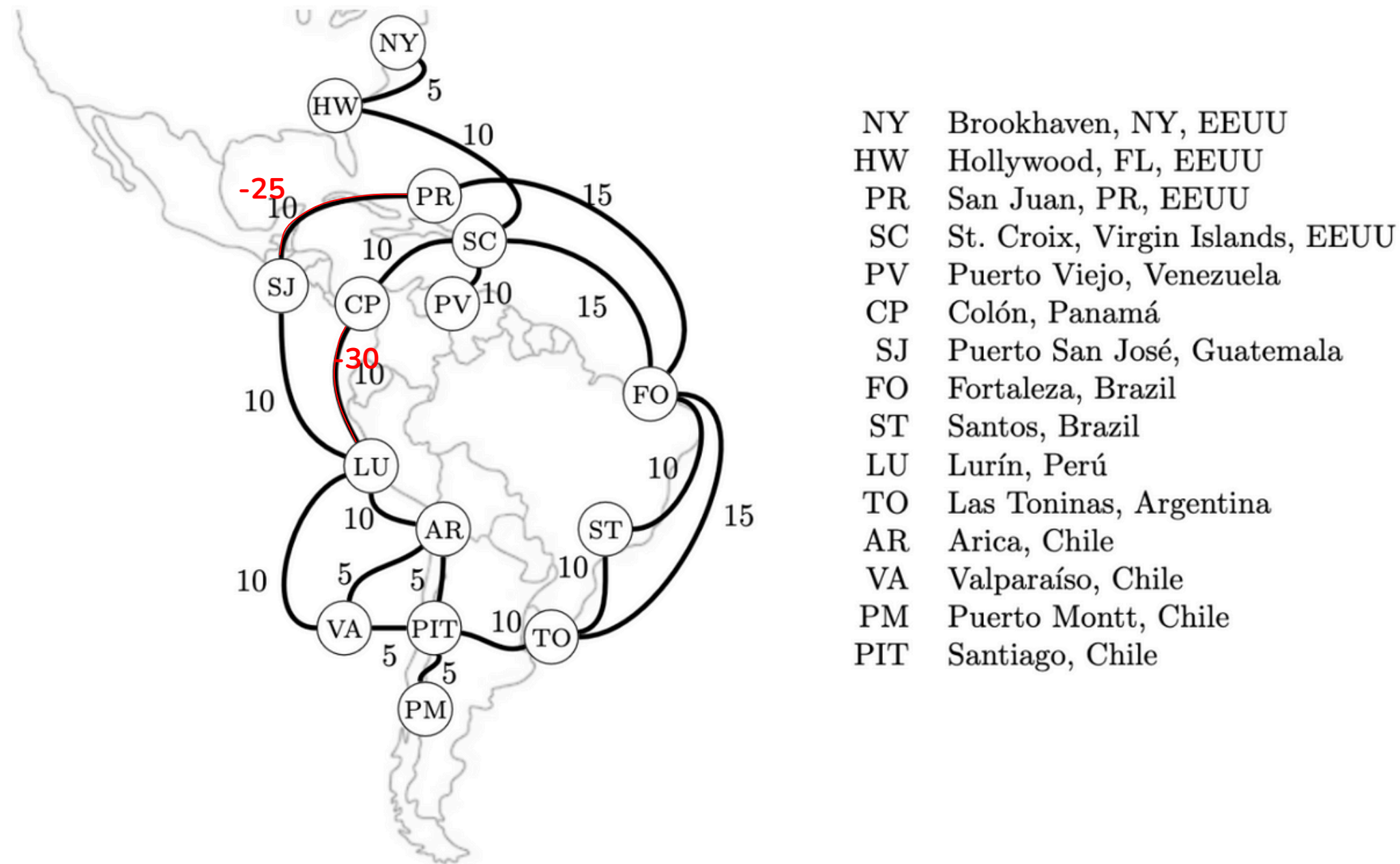
I3 2023-2

La red de fibra óptica terrestre y submarina que conecta nuestros computadores puede verse como una grafo no dirigido. En este contexto, se pueden utilizar los protocolos OSPF y RIP para determinar las rutas a utilizar. Considere el siguiente fragmento de la red, donde se indican las latencias como costos en aristas.



I3 2023-2

(c) [2 ptos.] Considere que se quiere privilegiar el uso de las aristas (LU,CP) y (SJ,PR) cambiando su peso por -30 y -25 respectivamente. Determine si con este cambio es posible usar el algoritmo de Bellman-Ford para determinar la ruta más barata de Santiago a Lurín. Justifique su respuesta.



I3 2023-2

(c) [2 ptos.] Considere que se quiere privilegiar el uso de las aristas (LU,CP) y (SJ,PR) cambiando su peso por -30 y -25 respectivamente. Determine si con este cambio es posible usar el algoritmo de Bellman-Ford para determinar la ruta más barata de Santiago a Lurín. Justifique su respuesta.

Solución.

Al modificar las aristas indicadas, existe un ciclo de costo negativo dado por

LU, CP, SC, FO, PR, SJ, LU

que es alcanzable desde PIT. Luego, el algoritmo de Bellman-Ford no se puede utilizar para obtener un camino de costo mínimo desde PIT hasta LU.

Puntajes.

1.0 por indicar la existencia de ciclo de costo negativo.

1.0 por indicar que no se puede usar Bellman-Ford debido a ello.

I3 2023-1 P2 A)

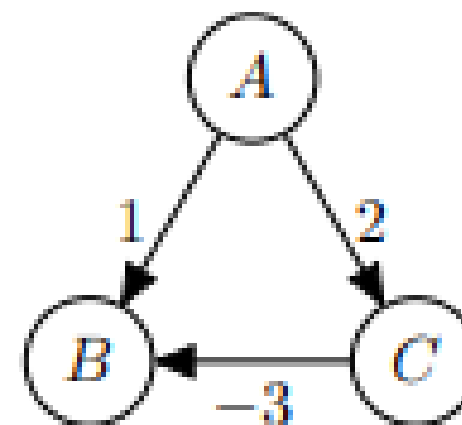
-) [3 ptos.] Demuestre que el algoritmo de Dijkstra no siempre es óptimo cuando el grafo dirigido sobre el que opera tiene costos negativos.

I3 2023-1 P2 A)

-) [3 ptos.] Demuestre que el algoritmo de Dijkstra no siempre es óptimo cuando el grafo dirigido sobre el que opera tiene costos negativos.

Solución.

Consideremos el grafo



El algoritmo de Dijkstra entrega como solución de A hasta B el camino A, B con costo 1, mientras que el óptimo real es el camino A, C, B con costo -1 .

I3 2023-1 P2 B)

(b) [3 ptos.] Sea G un grafo dirigido con costos.

(i) [2 ptos.] Proponga el pseudocódigo de un algoritmo que entregue una lista con los nodos de algún ciclo de costo negativo, en caso que G tenga tal ciclo. En caso contrario, retorna una lista vacía.

I3 2023-1 P2 B)

3 ptos.] Sea G un grafo dirigido con costos.

- (i) [2 ptos.] Proponga el pseudocódigo de un algoritmo que entregue una lista con los nodos de algún ciclo de costo negativo, en caso que G tenga tal ciclo. En caso contrario, retorna una lista vacía

Solución.

Utilizando como estructura una cola FIFO implementada como lista ligada, los métodos son

```
NegativeCycle( $s$ ):
1   for  $u \in V$  :
2        $d[u] \leftarrow \infty$ ;  $\pi[u] \leftarrow \emptyset$ 
3    $d[s] \leftarrow 0$ 
4   for  $k = 1 \dots |V| - 1$  :
5       for  $(u, v) \in E$  :
6           if  $d[v] > d[u] + cost(u, v)$  :
7                $d[v] \leftarrow d[u] + cost(u, v)$ 
8                $\pi[v] \leftarrow u$ 
9   for  $(u, v) \in E$  :
10      if  $d[v] > d[u] + cost(u, v)$  :
11           $L \leftarrow$  lista con elemento  $v$ 
12           $current \leftarrow u$ 
13          while  $current \neq v$  :
14              agregar a  $L$  el nodo  $current$ 
15               $current \leftarrow \pi[current]$ 
16  return Lista vacía
```

Notemos que el algoritmo es esencialmente `ValidBellmanFord`, al cual se le cambia su retorno booleano. En caso que exista ciclo negativo (resultado `false` del algoritmo visto en clases), se construye la lista visitando ancestros del nodo que genera el ciclo. En caso contrario, se retorna lista vacía.

Ahora, este método solo indica ciclo negativo desde una fuente. Si existe un ciclo negativo, pero no es alcanzable desde un nodo específico, tenemos que asegurarnos de detectarlo. Para esto, se puede ejecutar este método desde todos los vértices. Es decir

```
NegativeCycleGlobal():
1   for  $u \in V$  :
2        $L \leftarrow$  NegativeCycle( $u$ )
3       if  $L \neq \emptyset$  :
4           return  $L$ 
5   return Lista vacía
```